DATA MINING

Complete Study Notes and Chatbot Knowledge Base

Amity Business School, Amity University Rajasthan

MBA (Business Analytics Specialisation) | Semester III | 2026

Course: Data Mining (Theoretical and Practical) | L-T-P-C = 3-0-2-2 | Tool: Weka 3

Textbook: Han, Kamber and Pei, Data Mining: Concepts and Techniques (3rd ed.)

How these notes are organised. The notes follow the five modules of the official syllabus. Every topic is written in the same pattern: definition, explanation, steps or formula, worked example and (where relevant) the Weka procedure. Each topic heading carries its section number and Bloom's level so that a question can be matched to the right part quickly.

Sources and honesty note. Concepts follow the prescribed textbook (Han, Kamber and Pei) and the standard Weka book (Witten, Frank, Hall). Datasets used in worked examples are the well-known weather (play-tennis) data and the nine-transaction AllElectronics market-basket example. The loan data cube in Module I is an invented teaching example, not real data. Weka menu and parameter names are given for Weka 3.8/3.9 and may differ slightly in other versions.

Contents. Section 0: Course profile. Module I: Introduction to Data Mining (L2). Module II: Knowledge Representation and Attribute-Oriented Analysis (L3). Module III: Data Mining Algorithms (L3): association rules, classification (1R, decision trees, covering rules), prediction (Bayesian, nearest neighbour, linear models). Module IV: Cluster Analysis (L3). Module V: Advanced Techniques: text mining, web mining, software and applications (L4, L5). Appendices: formula sheet, Weka quick reference, glossary, frequently asked exam questions.

Section 0: Course Profile (Data Mining, MBA Semester III)

0.1 Course details

Field	Detail
Course	Data Mining (Theoretical and Practical), Amity Business School, MBA, Semester III, 2026
Specialisation	Business Analytics
L-T-P-C	3-0-2-2 (Lecture-Tutorial-Practical-Credits). Version NEP-V-1.1
Course code	Not yet allotted by the Board of Studies when the profile was prepared
Course coordinator	Yash Yadav, Block F, 2nd Floor, Amity Business School
Pre-requisites	Basic statistics and spreadsheet operations helpful but not mandatory. Co-requisites: Nil
Tool	Weka 3 (laboratory sessions)
Nature of course	A methods course. Preprocessing is taught before any algorithm because most practical difficulty lies in getting data into a state where a technique means something.

0.2 Course objectives

- Build a working understanding of data mining's conceptual foundations and its relationship to machine learning, DBMS and OLAP.
- Develop competence in preprocessing raw data: cleaning, transformation, reduction and discretisation.
- Apply core techniques (association rule mining, classification, prediction, clustering) using an industry-relevant tool (Weka).
- Evaluate mined patterns critically against business relevance and interestingness.
- Extend these techniques to advanced applications such as text mining and web mining.

0.3 Course outcomes and Bloom's levels

CO	Statement	Bloom
CO1	Understand and explain the fundamental concepts, processes and applications of data mining along with related technologies.	L2 (Understand)
CO2	Preprocess and transform raw data to generate suitable input for various data mining techniques and tools.	L2 (Understand)
CO3	Apply and evaluate major data mining techniques such as association rule mining, classification, prediction and clustering using tools like Weka.	L3 (Apply)
CO4	Evaluate complex data types and extract meaningful patterns for business decision-making through advanced data mining applications.	L5 (Evaluate)

Module	Title	Bloom's level	Related CO
I	Introduction to Data Mining	L2	CO1, CO2
II	Knowledge Representation and Attribute-Oriented Analysis	L3	CO2
III	Data Mining Algorithms	L3	CO3
IV	Cluster Analysis: Concepts and Methods	L3	CO3
V	Advanced Techniques: Data Mining Software and Applications	L4, L5	CO4

0.4 Assessment

Total 100 marks: Continuous Assessment (formative) 50 and End Semester Assessment (summative) 50. Component weightage in percent: Attendance 5, Class Test 25, End Semester Examination 50, External

Practical 5, Project 10, Internal Practical 5.

Task	Marks	Timing
Internal Practical (Weka lab record)	5	Continuous, weeks 3-9
Class Test	25	Week 8
Project (capstone mining exercise)	10	Weeks 9-10
External Practical (viva on project and lab work)	5	Week 10
Attendance (minimum normally 75%)	5	Last week of teaching
End Semester Examination	50	As per university schedule

Practical and project are judged on preprocessing rigour (data cleaned, transformed, discretised with justification), tool proficiency (using Weka and interpreting rules, trees and clusters in plain business language), problem framing, technique selection, and communication of findings at the viva. Plagiarism is screened by software; dataset sources and adapted code must be acknowledged.

0.5 Fifteen-week plan

Week	Topic	CO
1	What is data mining? Related technologies, goals and stages	CO1
2	Knowledge representation methods; data warehouse and DBMS; OLAP operations	CO1
3	Data cleaning, transformation, reduction; installing Weka 3; filters	CO1
4	Task-relevant data, background knowledge, interestingness measures, visualisation (Weka lab)	CO2
5	Attribute-oriented analysis: generalisation, relevance, class comparison (Weka lab)	CO2
6	Statistical measures and class comparison using filters; Class Test preparation	CO2
7	Association rule mining: item sets, rule generation, correlation analysis (Weka lab)	CO3
8	Classification: 1R, decision trees, covering rules (Weka lab). Class Test	CO3
9	Prediction: Bayesian classification, Bayesian networks, instance-based methods (Weka lab)	CO3
10	Cluster analysis: partitioning methods, k-means, EM (Weka lab)	CO3
11	Hierarchical, density-based, grid-based clustering; Cobweb (Weka lab)	CO3
12	Text mining, keyword extraction, Bayesian classification of text	CO4
13	Web mining; knowledge extraction from web pages; data mining software	CO4
14	Project development; lab record consolidation	CO3, CO4
15	Project presentation; viva-voce; external practical	CO4

0.6 Reference books

- Text: Han, J., Kamber, M., and Pei, J. (2011). Data Mining: Concepts and Techniques, 3rd ed., Morgan Kaufmann.
- Reference: Berson and Smith, Data Warehousing, Data Mining and OLAP (Tata McGraw-Hill); Marakas, Modern Data Warehousing, Mining and Visualization (Pearson); Berry and Linoff, Data Mining Techniques; Sharma, Data Mining, Data Warehousing and OLAP; Gupta, Data Mining with Case Studies; Ponniah, Data Warehousing Fundamentals.

MODULE I: Introduction to Data Mining (Bloom's level L2)

1.1 What is Data Mining?

Definition. Data mining is the process of discovering interesting, previously unknown and potentially useful patterns and knowledge from large amounts of data stored in databases, data warehouses or other information repositories. It is also called knowledge mining from data, knowledge discovery in databases (KDD), knowledge extraction, data/pattern analysis, data archaeology and data dredging.

Why it is needed. Organisations collect huge volumes of data (transactions, web logs, sensors) but are 'data rich and information poor'. Manual analysis cannot scale, so automated methods are needed to turn data into knowledge that supports decisions.

Key characteristics. The extracted patterns are (a) non-trivial, (b) implicit rather than obvious, (c) previously unknown, and (d) potentially useful for decisions.

What is not data mining. Looking up a phone number, running a simple SQL query, computing totals, or using a search engine to find a known page are ordinary retrieval or query processing. Finding that customers who buy product A tend to buy B, or segmenting customers into groups, is data mining.

Kinds of data that can be mined. Relational databases, data warehouses, transactional data, spatial data, time-series data, text, multimedia, web data and data streams.

1.2 Related Technologies: Machine Learning, DBMS, OLAP, Statistics

Technology	What it is	Relationship to data mining
Machine Learning (ML)	Field of algorithms that improve with experience by learning from data (supervised, unsupervised, reinforcement).	Supplies many of the algorithms (decision trees, clustering, neural networks). Data mining applies them to large, real, often messy business databases, with emphasis on understandable patterns and on the whole process including preprocessing.
DBMS (Database Management System)	Software for storing, organising and retrieving data efficiently (for example relational databases queried with SQL).	Provides the data and storage. A DBMS answers known queries; data mining discovers unknown patterns. Mining often reads from databases or warehouses.
OLAP (Online Analytical Processing)	Interactive, multidimensional analysis of summarised data using operations such as roll-up, drill-down, slice, dice and pivot.	OLAP is user-driven and verifies hypotheses ('what were sales by region?'). Data mining is discovery-driven and finds patterns automatically. OLAP and warehouses are often the source and complement of mining (OLAM, online analytical mining).
Statistics	Collection, analysis, interpretation and presentation of data using models and hypothesis tests.	Provides foundations (sampling, regression, correlation, probability, hypothesis testing). Statistics is typically hypothesis-driven on small samples; data mining works on very large data with less assumption and more automation.

Data mining versus OLAP versus query

Aspect	SQL query / DBMS	OLAP	Data mining
Purpose	Retrieve specific known data	Summarise and explore along dimensions	Discover hidden patterns and predict
Driven by	User question	User hypothesis (verification)	Algorithm (discovery)
Output	Records	Aggregates and cubes	Rules, trees, clusters, models

1.3 Data Mining Goals

Data mining has two broad high-level goals: **description** and **prediction**.

Aspect	Descriptive mining	Predictive mining
Goal	Characterise the general properties of existing data; find human-interpretable patterns	Use current data to make predictions about unseen or future data
Typical tasks	Clustering, association rules, summarisation, characterisation and discrimination	Classification, regression (numeric prediction), forecasting, anomaly detection
Example	Which products are bought together? What customer segments exist?	Will this customer churn? What will next month's demand be?

Other practical goals: summarise large data, classify items into known groups, group similar items, forecast values, detect outliers or fraud, and support decision-making.

1.4 Stages of the Data Mining Process (KDD Process)

Knowledge discovery is an iterative sequence of steps. Data mining proper is one step (some authors use 'data mining' for the whole process).

1. **Data cleaning:** remove noise and inconsistent data, handle missing values.
2. **Data integration:** combine data from multiple sources into a coherent store.
3. **Data selection:** retrieve the data relevant to the analysis task.
4. **Data transformation:** convert data into forms suitable for mining (normalisation, aggregation, generalisation).
5. **Data mining:** apply intelligent methods to extract patterns.
6. **Pattern evaluation:** identify the truly interesting patterns using interestingness measures.
7. **Knowledge presentation:** present the knowledge to users using visualisation and representation techniques.

Steps 1-4 are often grouped as **data preprocessing**. The process is iterative: results of later steps can send the analyst back to earlier steps.

Alternative view: CRISP-DM

Industry often uses CRISP-DM: Business understanding, Data understanding, Data preparation, Modelling, Evaluation, Deployment. It stresses framing the business problem first and deploying results at the end.

1.5 Data Mining Techniques (Kinds of Patterns)

- **Concept/class description (characterisation and discrimination):** summarise a target class (for example 'graduate students') and compare it with other classes.
- **Association analysis:** find items that occur together (market basket analysis). Output: association rules such as 'bread and butter implies milk'.
- **Classification:** learn a model from labelled examples to assign class labels to new data (decision trees, rules, Naive Bayes, nearest neighbour).
- **Prediction / regression:** predict a numeric value (sales, price) from other attributes.
- **Cluster analysis:** group objects into clusters of similar items without predefined labels (k-means, hierarchical).
- **Outlier (anomaly) analysis:** find objects that do not fit the general pattern (fraud detection).
- **Trend and evolution analysis:** time-series and sequential patterns.

Supervised versus unsupervised. Supervised learning (classification, regression) uses labelled training data. Unsupervised learning (clustering, association) has no class labels. Semi-supervised uses a few labelled and many unlabelled examples.

1.6 Knowledge Representation Methods

Mined knowledge must be expressed in a form people and programs can use. Main forms:

- **Tables / decision tables:** simplest, a lookup table of attribute combinations and outcomes.
- **Linear models:** a numeric formula such as y = w0 + w1*x1 + w2*x2.
- **Decision trees:** internal nodes test an attribute, branches are outcomes, leaves give classes. Easy to read and convert to rules.
- **Classification rules:** IF-THEN rules, for example IF outlook = sunny AND humidity = high THEN play = no. Can be a decision list (ordered) or an unordered rule set.
- **Association rules:** rules between any attributes or items, such as bread AND butter => milk [support, confidence].
- **Rules with exceptions:** a general rule plus exceptions to it, giving compact, human-like knowledge.
- **Rules involving relations:** rules that compare attributes with each other (for example width > height) rather than with constants.
- **Instance-based representation:** the stored examples themselves, with a distance function (nearest neighbour).
- **Clusters:** groupings of instances: exclusive, overlapping, probabilistic, or hierarchical (dendrogram).

1.7 Applications: The Weather Data Example

The **weather (play-tennis) dataset** is the standard teaching example used throughout this course and in Weka. It has 14 instances (days), four predictor attributes and a class. Goal: decide whether to play outdoors (play = yes/no).

No.	Outlook	Temperature	Humidity	Windy	Play
1	sunny	hot	high	false	no
2	sunny	hot	high	true	no
3	overcast	hot	high	false	yes
4	rainy	mild	high	false	yes
5	rainy	cool	normal	false	yes
6	rainy	cool	normal	true	no
7	overcast	cool	normal	true	yes
8	sunny	mild	high	false	no
9	sunny	cool	normal	false	yes
10	rainy	mild	normal	false	yes
11	sunny	mild	normal	true	yes
12	overcast	mild	high	true	yes
13	overcast	hot	normal	false	yes
14	rainy	mild	high	true	no

Class counts: play = yes in 9 instances, play = no in 5 instances. Attribute values: outlook (sunny 5, overcast 4, rainy 5), temperature (hot 4, mild 6, cool 4), humidity (high 7, normal 7), windy (false 8, true 6). All attributes are nominal in this version (weather.nominal.arff). A second version (weather.numeric.arff) keeps temperature and humidity as numbers.

Example knowledge that can be mined (decision list): IF outlook = overcast THEN play = yes. IF humidity = normal AND windy = false THEN play = yes. IF outlook = sunny AND humidity = high THEN play = no. IF outlook = rainy AND windy = true THEN play = no.

Business applications of data mining (detailed in Module V): retail market-basket analysis, customer segmentation and churn prediction, credit scoring and fraud detection, demand forecasting and inventory optimisation in supply chains, predictive maintenance in manufacturing, recommendation systems, healthcare diagnosis support.

1.8 Data Warehouse and DBMS

Data warehouse definition (W. H. Inmon). A data warehouse is a subject-oriented, integrated, time-variant and non-volatile collection of data in support of management's decision-making process.

- **Subject-oriented:** organised around major subjects (customer, product, sales) instead of day-to-day operations.
- **Integrated:** data from many heterogeneous sources is cleaned and made consistent (naming, units, encoding).
- **Time-variant:** holds historical data (for example the last 5-10 years); every key has a time element.
- **Non-volatile:** data is loaded and read, not updated or deleted in the transactional sense.

OLTP (operational DBMS) versus OLAP (data warehouse)

Feature	OLTP (operational database)	OLAP (data warehouse)
Users	Clerks, clients, IT staff	Managers, analysts, executives
Function	Day-to-day operations	Decision support and analysis
Data	Current, detailed, flat relational	Historical, summarised, multidimensional
Design	Application-oriented (ER model)	Subject-oriented (star/snowflake)
Access	Short atomic read/write transactions	Mostly complex read-only queries
Size	Megabytes to gigabytes	Gigabytes to terabytes and more
Metric	Transaction throughput	Query throughput and response time

Warehouse architecture

Three tiers: (1) bottom tier is the warehouse database server fed by ETL (extract, transform, load) from operational sources; (2) middle tier is the OLAP server (ROLAP, MOLAP or HOLAP); (3) top tier is the front-end (query, reporting, analysis and data mining tools). Models: enterprise warehouse, data mart (a department-level subset), and virtual warehouse (views over operational data). A metadata repository describes the warehouse contents.

1.9 Multidimensional Data Model

A warehouse is modelled as a **data cube** that lets data be viewed in multiple dimensions.

- **Dimensions:** the perspectives or entities about which an organisation wants records (time, location, product, customer). Each has a dimension table.
- **Fact table:** stores the numeric **measures** (for example amount sold, loan amount) and keys to the dimension tables.
- **Cuboid:** a cube at one level of summarisation. The base cuboid has the lowest level of detail; the apex cuboid (all dimensions summarised) is a single total value. For n dimensions with no hierarchies there are 2^n cuboids (the cuboid lattice). With hierarchies the number is the product of (levels + 1) over all dimensions.
- **Concept hierarchy:** mapping from low-level to higher-level concepts, for example day < month < quarter < year, or street < city < state < country.

Schemas for multidimensional data

Schema	Description
Star schema	One central fact table linked directly to one denormalised table per dimension. Simple and fast; some redundancy.
Snowflake schema	Variant of star where dimension tables are normalised into several related tables. Less redundancy, more joins.
Fact constellation (galaxy)	Multiple fact tables share dimension tables. Used for several related subjects (for example sales and shipping).

Measure types

Distributive (sum, count, min, max: can be computed from partial results), **algebraic** (average, standard deviation: computed from a fixed number of distributive measures), **holistic** (median, mode, rank: no fixed-size partial summary).

1.10 OLAP Operations (Loan Dataset Example)

OLAP operations are the interactive manipulations of a data cube: **roll-up**, **drill-down**, **slice**, **dice** and **pivot** (rotate).

- **Roll-up (drill-up):** aggregate data by climbing up a concept hierarchy or by removing a dimension (for example city to country, or total over all loan types).
- **Drill-down:** the reverse; move from summary to more detail (quarter to month) or add a dimension.
- **Slice:** select a single value on one dimension, producing a sub-cube with one fewer dimension (for example time = Q1).
- **Dice:** select values on two or more dimensions, producing a sub-cube (for example time in {Q1, Q2} AND city = Delhi AND type = Home).
- **Pivot (rotate):** rotate the axes of the view to see the data from a different perspective (swap rows and columns).
- Other operations: drill-across (query across several fact tables) and drill-through (go down to the back-end relational detail).

Illustrative loan data cube (invented teaching numbers, in lakh rupees)

Dimensions: Time (quarter), Location (city), Loan type. Measure: total loan amount sanctioned.

Quarter	City	Loan type	Amount
Q1	Jaipur	Home	50
Q1	Jaipur	Car	20
Q1	Delhi	Home	80
Q1	Delhi	Car	30
Q2	Jaipur	Home	60
Q2	Jaipur	Car	25
Q2	Delhi	Home	90
Q2	Delhi	Car	35

Grand total (apex cuboid) = 50+20+80+30+60+25+90+35 = 390. Q1 total = 180, Q2 total = 210.

Roll-up on Location (city to all cities): Q1 Home = 130, Q1 Car = 50, Q2 Home = 150, Q2 Car = 60.

Roll-up on Loan type (remove the loan-type dimension): Q1 Jaipur = 70, Q1 Delhi = 110, Q2 Jaipur = 85, Q2 Delhi = 125.

Slice (Time = Q1): Jaipur-Home 50, Jaipur-Car 20, Delhi-Home 80, Delhi-Car 30. The result is a 2-D sub-cube (city by loan type).

Dice (city = Delhi AND type = Home, quarters Q1 and Q2): Q1 = 80, Q2 = 90.

Pivot: a table with quarters as rows and cities as columns is rotated so that cities become rows and quarters columns. The numbers do not change, only the arrangement.

Drill-down: the opposite of roll-up, for example from quarter to month, which needs the cube to store monthly data.

OLAP server types

Type	Meaning	Pros and cons
ROLAP	Relational OLAP: uses an extended relational DBMS; cube operations translated to SQL	Scales to very large data; slower than MOLAP
MOLAP	Multidimensional OLAP: uses array-based multidimensional storage	Fast indexing and retrieval; poor with sparse data; limited size
HOLAP	Hybrid OLAP: detailed data in relational store, aggregates in multidimensional arrays	Balance of scalability and speed

1.11 Data Preprocessing

Why preprocess? Real-world data is **incomplete** (missing values), **noisy** (errors and outliers) and **inconsistent** (different codes or names). 'Garbage in, garbage out': low-quality data gives low-quality mining results. Data quality dimensions: accuracy, completeness, consistency, timeliness, believability, interpretability. Major tasks: data cleaning, integration, transformation, reduction, and discretisation.

1.11.1 Data Cleaning

Missing values. Methods:

1. Ignore (delete) the tuple. Usually done when the class label is missing; poor when many values are missing.
2. Fill in manually. Tedious and often infeasible.
3. Fill with a global constant such as 'Unknown'. Simple but the algorithm may treat it as a real value.
4. Fill with the attribute mean (numeric) or median (if skewed).
5. Fill with the attribute mean or median of all samples belonging to the same class.
6. Fill with the most probable value, predicted by regression, a Bayesian formula or a decision tree. Most accurate and most popular among advanced methods.

Noisy data. Noise is random error or variance in a measured variable. Smoothing techniques:

- **Binning:** sort the data, partition into bins (equal-width or equal-frequency/equal-depth), then smooth by bin means, bin medians or bin boundaries. It smooths using the 'neighbourhood' of values.
- **Regression:** fit the data to a function (linear or multiple regression) and use fitted values.
- **Clustering:** detect outliers as values falling outside clusters.
- **Combined computer and human inspection:** the computer flags suspicious values for a person to check.

Binning example. Sorted prices: 4, 8, 15, 21, 21, 24, 25, 28, 34. Equal-depth (3 values per bin): Bin 1 = 4, 8, 15; Bin 2 = 21, 21, 24; Bin 3 = 25, 28, 34. **Smoothing by bin means:** bin means are 9, 22, 29, so Bin 1 = 9, 9, 9; Bin 2 = 22, 22, 22; Bin 3 = 29, 29, 29. **Smoothing by bin boundaries:** each value moves to the nearer of the bin minimum or maximum: Bin 1 = 4, 4, 15; Bin 2 = 21, 21, 24; Bin 3 = 25, 25, 34. Equal-width binning would instead divide the range into intervals of equal size: width = (max - min)/number of bins = (34 - 4)/3 = 10, giving intervals 4-14, 14-24, 24-34.

Inconsistent data and data cleaning as a process. Inconsistencies (for example age = 42 but birthday = 03/07/2010, or rating codes '1,2,3' versus 'A,B,C') are fixed using external references or knowledge engineering tools. Data cleaning is a two-step iterative process: **discrepancy detection** (using metadata,

field overloading checks, uniqueness and null rules; data scrubbing and data auditing tools) and **data transformation** (data migration and ETL tools).

1.11.2 Data Integration

Combines data from multiple sources into one coherent store. Issues:

- **Entity identification problem:** matching the same real-world entity across sources (customer_id versus cust_number); use metadata.
- **Redundancy:** an attribute is redundant if it can be derived from others; redundancy is detected by correlation analysis. For nominal data use the chi-square test; for numeric data use the correlation coefficient and covariance.
- **Tuple duplication:** duplicate records from denormalised tables.
- **Data value conflict:** the same attribute has different values or units (metric versus imperial, different currencies).

```
Correlation coefficient (Pearson) between numeric attributes A and B:
  r(A,B) = sum[(a_i - mean_A)(b_i - mean_B)] / (n * sd_A * sd_B)
  r > 0 : positively correlated; r = 0 : no linear correlation; r < 0 : negative
  If |r| is high, one of the two attributes may be removed as redundant.
Chi-square for nominal attributes (observed vs expected counts):
  chi2 = sum[(observed - expected)^2 / expected]
  expected(cell) = (row total * column total) / grand total
  Large chi2 (above the critical value) => attributes are correlated (not independent).
```

1.11.3 Data Transformation

Convert data into forms appropriate for mining. Strategies:

- **Smoothing:** remove noise (binning, regression, clustering).
- **Aggregation:** summary operations, for example daily sales rolled into monthly sales.
- **Generalisation:** replace low-level data with higher-level concepts using concept hierarchies (city to country).
- **Normalisation:** scale values into a small specified range so that no attribute dominates distance calculations.
- **Attribute (feature) construction:** build new attributes from existing ones (area = width * height).
- **Discretisation:** replace numeric values by interval or concept labels.

Normalisation methods

```
Min-max normalisation to [new_min, new_max]:
  v' = (v - min) / (max - min) * (new_max - new_min) + new_min
Z-score (zero-mean) normalisation:
  v' = (v - mean) / standard_deviation
Decimal scaling:
  v' = v / 10^j      where j is the smallest integer such that max(|v'|) < 1
```

Example. Income ranges from 12,000 to 98,000. Min-max to [0,1] for v = 73,600: (73600 - 12000)/(98000 - 12000) = 61600/86000 = 0.716. Z-score with mean = 54,000 and standard deviation = 16,000: (73600 - 54000)/16000 = 1.225. Decimal scaling when the largest absolute value is 986: j = 3, so -986 becomes -0.986.

1.11.4 Data Reduction

Obtain a reduced representation that is much smaller but produces the same (or almost the same) analytical results.

- **Data cube aggregation:** aggregate data to the level needed (quarterly instead of daily).
- **Attribute subset selection:** remove irrelevant or redundant attributes. Methods: stepwise forward selection, stepwise backward elimination, combination of both, and decision tree induction (attributes not in the tree are dropped).

- **Dimensionality reduction:** transform to a smaller space, for example principal component analysis (PCA) and wavelet transforms.
- **Numerosity reduction:** replace data by smaller representations. Parametric: regression and log-linear models. Non-parametric: histograms, clustering, sampling.
- **Data compression:** lossless (original recoverable) or lossy (approximation only).

Sampling types: simple random sample without replacement (SRSWOR), simple random sample with replacement (SRSWR), cluster sample, and stratified sample (sample taken from each stratum or class to preserve distribution).

1.11.5 Discretisation and Concept Hierarchies

Discretisation reduces the number of values of a continuous attribute by dividing its range into intervals; interval labels replace the actual values. It can be supervised (uses class information) or unsupervised, and top-down (splitting) or bottom-up (merging). Discretisation produces a concept hierarchy of the numeric attribute (for example age: 0-10, 11-20 ... to young, middle-aged, senior).

Methods for numeric data:

- **Binning:** equal-width or equal-frequency; unsupervised and sensitive to outliers.
- **Histogram analysis:** equal-width or equal-frequency histograms; unsupervised.
- **Cluster analysis:** clusters become intervals.
- **Entropy-based (decision-tree) discretisation:** supervised; choose the split point that minimises class entropy, then recurse (this is the idea behind Fayyad-Irani MDL discretisation in Weka's supervised Discretize filter).
- **Chi-square (ChiMerge) discretisation:** bottom-up; merge adjacent intervals with similar class distributions.
- **Intuitive partitioning (3-4-5 rule):** splits a range into 3, 4 or 5 relatively equal 'natural' intervals based on the most significant digit: if the interval covers 3, 6, 7 or 9 distinct values at the most significant digit, split into 3 intervals; if 2, 4 or 8, split into 4; if 1, 5 or 10, split into 5.

Concept hierarchy generation for nominal data: (1) specify a partial order of attributes explicitly at schema level (street < city < state < country); (2) specify a portion of a hierarchy by explicit data grouping; (3) specify a set of attributes without order and let the system order them automatically using the number of distinct values (the attribute with the most distinct values is placed at the lowest level; the one with the fewest is at the top); (4) specify only a partial set of attributes.

1.12 Installing Weka 3 and Experiments with Filters

Weka (Waikato Environment for Knowledge Analysis) is a free, open-source Java workbench from the University of Waikato, New Zealand, with a collection of preprocessing tools, learning algorithms and visualisation. Download Weka 3 from the University of Waikato website and install it (Windows installers usually bundle Java; on other systems install Java first and run the jar). Start Weka and open the **Weka GUI Chooser**.

Application	Use
Explorer	Main interface with tabs: Preprocess, Classify, Cluster, Associate, Select attributes, Visualize
Experimenter	Run and compare algorithms on several datasets statistically
KnowledgeFlow	Drag-and-drop data-flow design of a mining process
Simple CLI	Command-line interface to Weka classes
Workbench	Combined single-window environment

ARFF file format. Weka's native format (Attribute-Relation File Format): a @relation line, one @attribute line per attribute (nominal values listed in braces or type numeric/string/date), then @data followed by comma-separated instances. Missing values are written as ?. Weka also reads CSV.

```
@relation weather
@attribute outlook {sunny, overcast, rainy}
@attribute temperature {hot, mild, cool}
@attribute humidity {high, normal}
@attribute windy {TRUE, FALSE}
@attribute play {yes, no}
@data
sunny,hot,high,FALSE,no
overcast,hot,high,FALSE,yes
```

Experiment: loading data and applying filters (Preprocess tab)

1. Explorer > Preprocess > Open file... and choose weather.nominal.arff (or your own ARFF/CSV).
2. Inspect the attribute list, the attribute statistics panel (type, missing, distinct, unique, min, max, mean, standard deviation) and the histogram at right (coloured by class).
3. Click Choose under Filter, pick a filter, click the filter name to set parameters, then press Apply. Use Undo to revert, and Save to write the result.

Filter (package path)	Purpose
unsupervised.attribute.ReplaceMissingValues	Replace missing values with the mean (numeric) or mode (nominal)
unsupervised.attribute.Discretize	Equal-width (default) or equal-frequency binning of numeric attributes (options: bins, useEqualFrequency, attributeIndices)
supervised.attribute.Discretize	Class-aware (entropy/MDL) discretisation
unsupervised.attribute.Normalize	Scale numeric attributes to [0,1] (min-max)
unsupervised.attribute.Standardize	Z-score: zero mean, unit standard deviation
unsupervised.attribute.Remove	Delete selected attributes
unsupervised.attribute.NumericToNominal / NominalToBinary	Change attribute types (needed for Apriori and some algorithms)
unsupervised.attribute.StringToWordVector	Convert string attributes to word-frequency vectors (text mining)
supervised.attribute.AttributeSelection	Attribute subset selection inside the filter framework
unsupervised.instance.Resample / supervised.instance.Resample	Random sampling of instances (supervised version can bias toward uniform class distribution)
unsupervised.instance.RemovePercentage / RemoveDuplicates / RemoveWithValues	Delete a percentage of, duplicate, or value-matching instances

Supervised versus unsupervised filters: supervised filters use the class attribute (must be set), unsupervised ones do not. **Attribute versus instance filters:** attribute filters modify columns, instance filters modify rows.

Lab: discretise numeric weather data. Open weather.numeric.arff. Choose unsupervised.attribute.Discretize, set bins = 3, attributeIndices = 2,3 (temperature, humidity), Apply. The numeric values are replaced by intervals such as '(-inf-69.5]', '(69.5-77.5]', '(77.5-inf)'. Use useEqualFrequency = true for equal-depth bins.

MODULE II: Data Mining Knowledge Representation and Attribute-Oriented Analysis (Bloom's level L3)

2.1 Task-Relevant Data

Definition. Task-relevant data is the portion of the database (or warehouse) that the user is interested in, specified before mining so that the algorithm does not waste effort on irrelevant data. A data mining query (for example in DMQL, the Data Mining Query Language) specifies it.

It includes: (1) the name of the database or data warehouse; (2) the relevant tables or data cubes and the conditions for selecting data; (3) the relevant attributes or dimensions; (4) grouping criteria; (5) the kind of knowledge to be mined (characterisation, discrimination, association, classification, prediction, clustering); (6) background knowledge (concept hierarchies); (7) interestingness measures and thresholds; (8) how to present discovered patterns.

Example. 'Describe the general characteristics of graduate students in Big University.' Task-relevant data: table student, attributes name, gender, major, birth_place, birth_date, residence, phone, GPA, condition status = graduate. Name and phone are usually irrelevant to characterisation.

2.2 Background Knowledge

Background knowledge is knowledge about the domain that guides the mining process and helps express discovered patterns at multiple abstraction levels. The most common form is the **concept hierarchy**: a sequence of mappings from low-level concepts to higher-level, more general concepts.

- **Schema hierarchy:** total or partial order among attributes in the schema (street < city < province < country).
- **Set-grouping hierarchy:** groups values into sets (age: 20-39 = young, 40-59 = middle-aged).
- **Operation-derived hierarchy:** derived by an operation such as parsing (email address to domain) or clustering.
- **Rule-based hierarchy:** defined by rules (a low-profit item is one with profit margin below 50 percent of average).

Other background knowledge: user beliefs, domain constraints and prior results. Concept hierarchies let us roll up (generalise) and drill down (specialise) to see knowledge at different levels.

2.3 Interestingness Measures

A data mining system can generate thousands of patterns; only a small fraction are interesting. A pattern is **interesting** if it is (1) easily understood by humans, (2) valid on new or test data with some degree of certainty, (3) potentially useful, and (4) novel. A pattern is also interesting if it validates a hypothesis the user wanted to confirm.

Measure	Meaning	Examples
Simplicity	How easy a pattern is to understand	Rule length, number of leaves in a tree
Certainty	Validity or trustworthiness of the pattern	Confidence of a rule: P(B given A). Classification reliability or accuracy
Utility	Usefulness (potential for action)	Support: fraction of data in which the pattern appears
Novelty	Not previously known, surprising	A pattern that contradicts expectations or is not implied by known rules

Objective measures are based on the structure of the pattern and statistics (support, confidence, lift, accuracy, information gain, chi-square). **Subjective measures** depend on the user and the business context (unexpectedness, actionability). Interestingness measures are used both to filter patterns after mining and to **prune the search** during mining (for example minimum support in Apriori). Course

objective: evaluate mined patterns against business relevance rather than treating algorithm output as automatically useful.

2.4 Representing Input Data and Output Knowledge

Input: instances, attributes and concepts

The input to a mining algorithm is a set of **instances** (examples, rows, records), each described by values of a fixed set of **attributes** (features, columns). The thing to be learnt is the **concept**; the learnt description is the **concept description**. In supervised learning one attribute is the **class**.

Attribute scale	Meaning	Operations	Example
Nominal (categorical)	Names or labels, no order	Equality only	outlook = sunny, overcast, rainy
Ordinal	Ordered categories, differences not meaningful	Order comparisons	temperature = hot > mild > cool; education level
Interval	Numeric with equal intervals, no true zero	Add, subtract	Temperature in Celsius, calendar year
Ratio	Numeric with a true zero	All arithmetic, ratios	Income, weight, age, sales amount

Practical points: **missing values** (marked ? in ARFF) may be meaningful or accidental; **inaccurate values** (typing errors, deliberate false data) should be checked; **denormalisation** (flattening several relations into one table) may be needed to get one table per instance; **numeric versus nominal** types must be set correctly, because many algorithms (for example Apriori) require nominal data.

Output: types of knowledge representation

Output forms: decision tables, linear models, decision trees, classification rules, association rules, rules with exceptions and relational rules, instance-based representation, and clusters (see section 1.6). Trees and rules are the most interpretable and are favoured when the business must explain a decision.

Weather example of output forms. Tree: outlook at the root; sunny leads to a humidity test; overcast leads to yes; rainy leads to a windy test. Rule: IF outlook = overcast THEN play = yes. Association rule: IF temperature = cool THEN humidity = normal. Cluster: days grouped by similar weather.

2.5 Visualisation Techniques

Visualisation presents data and patterns graphically so humans can spot structure, outliers and relationships. It is used for data exploration before mining, for presenting results, and for interactive mining.

- **Bar chart / histogram:** distribution of a single attribute.
- **Scatter plot:** relationship between two numeric attributes; colour by class to see separation. A scatter plot matrix shows all pairs.
- **Box plot:** five-number summary and outliers.
- **Line chart:** trends over time. **Pie chart:** proportions.
- **Data cube / cross-tab views:** results of OLAP operations.
- **Tree and cluster visualisation:** decision tree diagrams, dendrograms, cluster plots.
- **High-dimensional techniques:** parallel coordinates, star plots, Chernoff faces, pixel-oriented displays.

Weka visualisation experiments

- Preprocess tab: select an attribute to see its histogram; set the class (Colour) so bars are split by class. Click Visualize All to see all histograms.
- Visualize tab: scatter plot matrix. Choose attributes for X and Y, use PlotSize and PointSize, and Jitter to separate overlapping nominal points. Click a point for instance details; use Select Instance

(rectangle, polygon, polyline) to subset data.

- Classify tab: right-click a result in the result list and choose Visualize tree, Visualize classifier errors, Visualize threshold curve (ROC) or Visualize margin curve.
- Cluster tab: right-click a result and choose Visualize cluster assignments.

2.6 Attribute-Oriented Induction (Attribute Generalisation)

Definition. Attribute-oriented induction (AOI) is a set-oriented, database-oriented method for concept description (data characterisation and comparison). It collects the task-relevant data with a relational query and then **generalises** it by replacing attribute values with higher-level concepts from concept hierarchies, merging identical tuples and accumulating counts. The result is a compact **generalised relation** (also called prime relation) that is converted into rules, tables or charts.

The generalisation algorithm

1. **Data focusing (initial relation):** collect the task-relevant data using a database query.
2. **Attribute removal:** remove an attribute if (a) it has a large number of distinct values and there is no generalisation operator (no concept hierarchy) for it, or (b) its higher-level concepts are expressed by other attributes.
3. **Attribute generalisation:** if an attribute has many distinct values and a generalisation operator exists, replace its values by their parent concepts in the hierarchy, repeatedly.
4. **Attribute generalisation threshold control:** keep generalising until the number of distinct values of the attribute is at or below a threshold (typically 2 to 8).
5. **Generalised relation threshold control:** limit the number of tuples in the final relation (typically 10 to 30); generalise further if it is exceeded.
6. **Count/aggregate accumulation:** when generalisation makes tuples identical, merge them and add their counts (a count attribute is added to the relation).
7. **Presentation:** display the result as a generalised relation, cross-tabulation, bar/pie chart, or as logical rules with quantitative information (t-weight, d-weight).

```
t-weight (typicality, for characteristic rules):
  t-weight(q_a) = count(q_a) / (sum of counts over all tuples in the target class)
d-weight (discrimination, for comparison rules):
  d-weight(q_a) = count(q_a in target class C_j) / (sum over all classes C_i of
                                                    count(q_a in C_i))
```

Example (characterisation). Target class: graduate students. After removing name and phone (many distinct values, no hierarchy), birth_place is generalised to country, birth_date to age range, residence to city/zone, GPA to {excellent, very good, good}, major to {science, engineering, business}. Suppose the generalised relation is: (Canada, science, 20-25, very good, count 16), (Canada, science, 25-30, excellent, count 22), (foreign, business, 20-25, excellent, count 18), (foreign, science, 25-30, excellent, count 25). Total count = 81. Characteristic rule: grad_student(x) => birth_place(x) = Canada [t-weight 38/81 = 46.9%] OR birth_place(x) = foreign [t-weight 43/81 = 53.1%].

Characterisation versus comparison. *Data characterisation* summarises the target class only. *Class comparison (discrimination)* compares the target class with one or more contrasting classes.

2.7 Attribute Relevance Analysis

Definition. Attribute relevance analysis identifies which attributes carry the most information for a task, so that irrelevant or weakly relevant attributes can be removed before mining. It reduces dimensionality, improves accuracy and speeds up mining.

General procedure (relevance analysis in concept description)

1. Data collection: collect target class and contrasting class data with queries.

2. Preliminary relevance analysis using a conservative AOI (generalise to a fairly low level) with attribute generalisation thresholds that retain enough detail.
3. Remove irrelevant or weakly relevant attributes using a relevance measure and a threshold.
4. Generate the concept description using AOI on the remaining attributes (generalise more strongly).

Relevance measures

Common measures: **information gain**, **gain ratio**, **Gini index**, chi-square, correlation coefficient and the uncertainty coefficient. Information gain is the most used.

```
Entropy (expected information) of data set D with m classes:
  Info(D) = - sum over i of p_i * log2(p_i)         p_i = fraction of D in class i
Expected information after splitting on attribute A with v values:
  Info_A(D) = sum over j of ( |D_j| / |D| ) * Info(D_j)
Information gain:
  Gain(A) = Info(D) - Info_A(D)
Higher gain => more relevant attribute.
```

Weather data. Info(D) with 9 yes and 5 no = -(9/14)log2(9/14) - (5/14)log2(5/14) = 0.940 bits. For outlook: sunny (5 instances: 2 yes, 3 no) Info = 0.971; overcast (4 instances: 4 yes, 0 no) Info = 0; rainy (5 instances: 3 yes, 2 no) Info = 0.971. Info_outlook = (5/14)(0.971) + (4/14)(0) + (5/14)(0.971) = 0.694. Gain(outlook) = 0.940 - 0.694 = 0.247. Similarly Gain(humidity) = 0.152, Gain(windy) = 0.048, Gain(temperature) = 0.029. Outlook is the most relevant attribute; temperature the least.

Weka: Select attributes tab

Open the Select attributes tab. Choose Attribute Evaluator = InfoGainAttributeEval (or GainRatioAttributeEval, CorrelationAttributeEval, ChiSquaredAttributeEval) and Search Method = Ranker, then Start. Weka lists attributes ranked by merit. For subset selection use CfsSubsetEval with BestFirst or GreedyStepwise, or WrapperSubsetEval with a classifier.

2.8 Class Comparison (Discrimination)

Definition. Class comparison (discriminant description) mines descriptions that distinguish a **target class** from one or more **contrasting classes**, for example comparing graduate and undergraduate students. The classes must be comparable (same attributes and level of abstraction).

Procedure

1. Data collection: retrieve target and contrasting class data using queries, and set up the attribute list.
2. Dimension relevance analysis (remove irrelevant attributes).
3. Synchronous generalisation: generalise the target class to the desired level, and generalise the contrasting classes to the same level, producing prime target class and prime contrasting class relations.
4. Presentation: show comparative tables, cross-tabs, charts, or discriminant rules with d-weights.

Example. Count of persons aged 20-25 with excellent GPA: graduates 20, undergraduates 80. d-weight for graduates = 20/(20+80) = 20%. A high d-weight means the value is a strong discriminator of the target class. Discriminant rule: IF age = 20-25 AND GPA = excellent THEN class = graduate [d-weight 20%].

Aspect	Characterisation	Class comparison
Aim	Summarise a single target class	Distinguish target class from contrasting classes
Data used	Target class only	Target and contrasting classes
Weight	t-weight (typicality)	d-weight (discrimination)
Rule form	Target => condition (necessary condition)	Condition => target (sufficient condition)

2.9 Statistical Measures in Large Databases

Descriptive statistics describe the central tendency and spread of data, detect outliers and guide preprocessing. Efficient computation needs distributive and algebraic measures (see 1.9).

Measures of central tendency

```
Mean (arithmetic):      mean = (x_1 + x_2 + ... + x_n) / n
Weighted mean:          sum(w_i * x_i) / sum(w_i)
Median:                 middle value of sorted data (average of two middle if n is even)
Mode:                   most frequent value (unimodal, bimodal, multimodal)
Midrange:               (max + min) / 2
Empirical relation (moderately skewed): mean - mode = 3 * (mean - median)
```

Measures of dispersion

```
Range = max - min
Quartiles: Q1 (25th percentile), Q2 (median), Q3 (75th percentile); IQR = Q3 - Q1
Five-number summary: min, Q1, median, Q3, max   (drawn as a box plot)
Outlier rule of thumb: value below Q1 - 1.5*IQR or above Q3 + 1.5*IQR
Variance:  s^2 = sum((x_i - mean)^2) / (n - 1)   (sample)   or / n  (population)
Standard deviation: s = square root of variance
```

Example. Data: 4, 8, 15, 21, 21, 24, 25, 28, 34. n = 9, mean = 180/9 = 20, median = 21, mode = 21, range = 34 - 4 = 30, Q1 = 11.5 (median of lower half 4, 8, 15, 21), Q3 = 26.5 (median of upper half 24, 25, 28, 34), IQR = 15. Outlier fences: 11.5 - 22.5 = -11 and 26.5 + 22.5 = 49, so no outliers.

Relationship measures

```
Covariance:  Cov(A,B) = sum((a_i - mean_A)(b_i - mean_B)) / n
Correlation: r = Cov(A,B) / (sd_A * sd_B)        range -1 to +1
Skewness: positive skew => mean > median > mode (long right tail)
```

Weka for statistics and class comparison. In the Preprocess tab, the selected-attribute panel gives minimum, maximum, mean and standard deviation for numeric attributes and counts for nominal ones. Colour by class to compare classes. Use filters (for example RemoveWithValues, or Resample) to extract the target and contrasting classes, then compare their statistics.

MODULE III: Data Mining Algorithms (Bloom's level L3)

3.1 Association Rule Mining

3.1.1 Motivation and terminology

Motivation. Market basket analysis: find which items customers buy together to support shelf layout, cross-selling, promotions, catalogue design and inventory planning. A classic story: 'customers who buy diapers also buy beer'. Association rule mining finds relationships among items in transactional databases; it is unsupervised and works on any attribute-value data (every attribute-value pair is treated as an item).

- **Item:** a single product or attribute-value pair (milk, outlook = sunny). I = set of all items.
- **Transaction:** a set of items bought together (a basket), with an ID. D = the set of all transactions.
- **Itemset:** a set of items; a **k-itemset** has k items. {milk, bread} is a 2-itemset.
- **Association rule:** an implication A => B where A (antecedent, premise) and B (consequent) are disjoint itemsets.
- **Support count (frequency, count) of an itemset:** number of transactions that contain it.
- **Support of a rule A => B:** fraction (or percentage) of transactions containing both A and B.
- **Confidence of A => B:** fraction of transactions containing A that also contain B (conditional probability P(B | A)). Called accuracy in some books.
- **Minimum support (min_sup) and minimum confidence (min_conf):** user thresholds.
- **Frequent (large) itemset:** an itemset whose support is at least min_sup.
- **Strong rule:** a rule that satisfies both min_sup and min_conf.

```
support(A => B)    = P(A and B) = count(A and B) / N     (N = number of transactions)
confidence(A => B) = P(B | A)    = count(A and B) / count(A)
lift(A => B)       = confidence(A => B) / P(B) = P(A and B) / (P(A) * P(B))
```

Two-step process: (1) find all frequent itemsets (computationally expensive); (2) generate strong association rules from the frequent itemsets (easy).

3.1.2 Generating item sets and rules efficiently: the Apriori algorithm

Apriori property (anti-monotone property of support). All non-empty subsets of a frequent itemset must also be frequent. Equivalently, if an itemset is infrequent, all its supersets are infrequent and need not be counted. This is the basis for pruning the search space.

Apriori algorithm: steps

1. Scan the database to count each item; keep those with support count at least min_sup count to form L1 (frequent 1-itemsets).
2. **Join step:** generate candidate k-itemsets C_k by joining L(k-1) with itself (join two (k-1)-itemsets that agree on their first k-2 items).
3. **Prune step:** delete any candidate in C_k that has an infrequent (k-1)-subset (Apriori property).
4. Scan the database to count the support of the remaining candidates; keep those that meet min_sup to form L_k.
5. Repeat steps 2 to 4, increasing k, until no new frequent itemsets (L_k is empty).
6. **Rule generation:** for each frequent itemset l, generate all non-empty proper subsets s; output the rule s => (l - s) if support_count(l) / support_count(s) >= min_conf.

Worked example (AllElectronics, 9 transactions, min support count = 2, min confidence = 70%)

TID	Items
T100	I1, I2, I5

TID	Items
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Pass 1. C1 counts: I1 = 6, I2 = 7, I3 = 6, I4 = 2, I5 = 2. All meet the minimum count of 2, so L1 = {I1}, {I2}, {I3}, {I4}, {I5}.

Pass 2. C2 (all pairs) with counts: {I1,I2} = 4, {I1,I3} = 4, {I1,I4} = 1, {I1,I5} = 2, {I2,I3} = 4, {I2,I4} = 2, {I2,I5} = 2, {I3,I4} = 0, {I3,I5} = 1, {I4,I5} = 0. L2 = {I1,I2}:4, {I1,I3}:4, {I1,I5}:2, {I2,I3}:4, {I2,I4}:2, {I2,I5}:2.

Pass 3. Join L2 with itself gives {I1,I2,I3}, {I1,I2,I5}, {I1,I3,I5}, {I2,I3,I5}, {I2,I3,I4}, {I2,I4,I5}. Prune: {I1,I3,I5} has subset {I3,I5} (not frequent), {I2,I3,I5} has {I3,I5}, {I2,I3,I4} has {I3,I4}, {I2,I4,I5} has {I4,I5}; all four are removed. C3 = {I1,I2,I3}, {I1,I2,I5}. Counts: {I1,I2,I3} = 2 (T800, T900), {I1,I2,I5} = 2 (T100, T800). L3 = both.

Pass 4. Joining L3 gives {I1,I2,I3,I5}; its subset {I1,I3,I5} is not frequent, so it is pruned. C4 is empty and the algorithm stops.

Rule generation from frequent itemset {I1, I2, I5} (support count 2). Non-empty proper subsets and rules:

Rule	Confidence calculation	Confidence	Strong at 70%?
I1 AND I2 => I5	2 / 4	50%	No
I1 AND I5 => I2	2 / 2	100%	Yes
I2 AND I5 => I1	2 / 2	100%	Yes
I1 => I2 AND I5	2 / 6	33%	No
I2 => I1 AND I5	2 / 7	29%	No
I5 => I1 AND I2	2 / 2	100%	Yes

With min_conf = 70% the strong rules are I1 AND I5 => I2, I2 AND I5 => I1 and I5 => I1 AND I2.

Improving the efficiency of Apriori

- **Hash-based itemset counting:** hash buckets let infrequent k-itemsets be discarded early.
- **Transaction reduction:** a transaction with no frequent k-itemsets cannot contain frequent (k+1)-itemsets, so skip it in later scans.
- **Partitioning:** any frequent itemset must be frequent in at least one partition; needs only two database scans.
- **Sampling:** mine a random sample with a lowered min_sup.
- **Dynamic itemset counting:** add new candidate itemsets at checkpoints during a scan.

Limitations of Apriori: may generate a huge number of candidates and needs repeated database scans. **FP-growth** avoids candidate generation: it compresses the database into a frequent-pattern tree (FP-tree) of frequent items sorted by descending frequency and mines it recursively by pattern-fragment growth, usually much faster.

3.1.3 Correlation analysis

High support and confidence do not guarantee an interesting rule: A => B may be misleading if B is common anyway. **Correlation (lift)** measures dependence between A and B.

```
lift(A,B) = P(A and B) / ( P(A) * P(B) )
  lift = 1 : A and B are independent (no correlation)
  lift > 1 : positively correlated (A makes B more likely)
  lift < 1 : negatively correlated
Other measures: leverage = P(A and B) - P(A)P(B);
conviction = (1 - P(B)) / (1 - confidence); chi-square test; all_confidence; cosine.
```

Example. Of 10,000 transactions, 6,000 contain computer games, 7,500 contain videos, and 4,000 contain both. Rule games => videos has support 4000/10000 = 40% and confidence 4000/6000 = 66.7%, which looks strong. But P(videos) = 75%, so lift = 0.40 / (0.60 * 0.75) = 0.89, which is below 1: buying games is actually negatively correlated with buying videos. The rule is misleading.

Weka: Associate tab

- Data must be **nominal**: discretise numeric attributes first (Discretize filter) or use NumericToNominal.
- Associate tab > Choose Apriori > Start. Weka's Apriori reports the number of cycles performed, the best rules found, and for each rule the antecedent and consequent with counts and metrics.
- Key parameters: **lowerBoundMinSupport** (default 0.1), **upperBoundMinSupport** (default 1.0), **delta** (default 0.05, the step by which support is decreased), **metricType** (confidence, lift, leverage, conviction), **minMetric** (default 0.9 for confidence), **numRules** (default 10), and **classIndex** / car (class association rules).
- Weka starts at the upper-bound support and keeps lowering min support by delta until numRules rules with the required metric are found or the lower bound is reached.
- FPGrowth is also available for binary (market-basket) data.

Lab. Load weather.nominal.arff, go to Associate, Apriori with default settings. A typical rule found: humidity = normal AND windy = false (4 instances) => play = yes (4), confidence 1.0. Try outlook = overcast => play = yes (4 of 4).

3.2 Classification

3.2.1 Basic learning / mining tasks

Classification builds a model (classifier) that describes data classes so that the class label of new objects can be predicted. It is **supervised learning**: training data comes with known class labels.

1. **Learning (model construction) step:** each training tuple has a class label; the algorithm builds a model (tree, rules or formula).
2. **Classification (model usage) step:** test data with known labels estimates the accuracy of the model; if accuracy is acceptable the model classifies new data whose label is unknown.

Classification versus prediction. Classification predicts categorical (discrete) class labels; prediction (numeric prediction or regression) models continuous-valued functions. Some books use 'prediction' for both.

Evaluating classifiers

```
Confusion matrix (two classes):         Predicted yes   Predicted no
                        Actual yes          TP              FN
                        Actual no           FP              TN
Accuracy  = (TP + TN) / (TP + TN + FP + FN);   Error rate = 1 - accuracy
Precision = TP / (TP + FP)          Recall (sensitivity, TP rate) = TP / (TP + FN)
F-measure = 2 * precision * recall / (precision + recall)
Specificity = TN / (TN + FP)
```

- **Holdout:** split into training (commonly two-thirds) and test (one-third) sets.

- **k-fold cross-validation:** split data into k folds; train on k-1 and test on 1, repeating k times; average the results (10-fold is standard; stratified in Weka).
- **Leave-one-out:** k = number of instances.
- **Bootstrap:** sampling with replacement.
- **Overfitting:** a model fits the training data too closely and performs poorly on new data. **Underfitting:** model too simple.

Other criteria: speed, robustness (noise and missing values), scalability and interpretability.

3.2.2 The 1R algorithm (One Rule)

Definition. 1R is a very simple classifier that generates a one-level decision tree: a set of rules that all test a single attribute. It often performs surprisingly well and serves as a baseline.

1. For each attribute, for each of its values, count how often each class appears.
2. Assign that value the most frequent class (make a rule: IF attribute = value THEN most frequent class).
3. Calculate the error rate of the rules for that attribute (number of instances not in the majority class).
4. Choose the attribute whose rules have the smallest total error and use its rule set.

Attribute	Rules	Errors	Total errors
outlook	sunny -> no; overcast -> yes; rainy -> yes	2/5, 0/4, 2/5	4/14
temperature	hot -> no*; mild -> yes; cool -> yes	2/4, 2/6, 1/4	5/14
humidity	high -> no; normal -> yes	3/7, 1/7	4/14
windy	false -> yes; true -> no*	2/8, 3/6	5/14

* tie between yes and no, chosen arbitrarily. Outlook and humidity tie at 4/14 (about 71% accuracy on training data); either may be chosen, for example the outlook rules. **Numeric attributes:** 1R discretises them by sorting values and creating intervals; to avoid overfitting (one interval per instance) each interval must contain at least a minimum number of majority-class instances (Weka OneR minBucketSize default 6). **Missing values** are treated as a separate attribute value.

Weka: Classify tab > Choose rules > OneR. Test options: Use training set, Supplied test set, Cross-validation (Folds = 10), Percentage split. Output shows the rule, correctly and incorrectly classified instances, kappa, errors, detailed accuracy by class and the confusion matrix.

3.2.3 Decision trees

Definition. A decision tree is a flowchart-like tree in which each internal node tests an attribute, each branch is an outcome of the test, and each leaf holds a class label. The topmost node is the root. Classification of a new instance follows the path from the root to a leaf.

Basic algorithm (ID3, C4.5, CART): greedy top-down recursive divide-and-conquer.

1. Start with all training tuples at the root.
2. If all tuples at a node are in the same class, make it a leaf with that class.
3. Otherwise choose the best splitting attribute using an attribute selection measure.
4. Create a branch for each outcome of the test and partition the tuples accordingly.
5. Recurse on each partition, never reusing a nominal attribute along a path.
6. Stop when (a) all tuples belong to the same class, (b) no attributes remain (use majority voting), or (c) no tuples remain.

Attribute selection measures

```
Information gain (ID3):  Gain(A) = Info(D) - Info_A(D)  (biased to many-valued)
Gain ratio (C4.5):       GainRatio(A) = Gain(A) / SplitInfo(A)
   SplitInfo(A) = - sum over j of (|D_j|/|D|) * log2(|D_j|/|D|)
Gini index (CART):       Gini(D) = 1 - sum of p_i^2
   Gini_A(D) = sum over j of (|D_j|/|D|) * Gini(D_j)
```

```
choose the split with the smallest Gini_A
(CART makes binary splits.)
```

Weather data, step by step. Gains at the root: outlook 0.247, humidity 0.152, windy 0.048, temperature 0.029, so outlook becomes the root. Gain ratio check for outlook: SplitInfo = -(5/14)log2(5/14) - (4/14)log2(4/14) - (5/14)log2(5/14) = 1.577, so GainRatio = 0.247/1.577 = 0.157. Gini(D) = 1 - (9/14)^2 - (5/14)^2 = 0.459. Branch overcast: all 4 are yes, so leaf = yes. Branch sunny (5 instances, 2 yes and 3 no): gains are humidity 0.971, temperature 0.571, windy 0.020, so test humidity: high = no (3 of 3), normal = yes (2 of 2). Branch rainy (5 instances, 3 yes and 2 no): windy gives pure splits: false = yes (3), true = no (2).

```
outlook = sunny    -> humidity = high   : no
                      humidity = normal : yes
outlook = overcast -> yes
outlook = rainy    -> windy = false     : yes
                      windy = true      : no
```

Numeric attributes (C4.5): sort values, consider split points (midpoints between adjacent values with different classes), test 'A <= split' versus 'A > split', choose the one with the highest gain. **Missing values:** C4.5 distributes the instance fractionally among branches in proportion to branch sizes.

Overfitting and tree pruning

- **Prepruning:** halt construction early (do not split if the measure falls below a threshold, or node is too small).
- **Postpruning:** build the full tree, then remove subtrees (replace by a leaf). C4.5 uses pessimistic error estimates (subtree replacement and subtree raising); CART uses cost-complexity pruning; reduced-error pruning uses a validation set.

Extracting rules from a tree: one IF-THEN rule per root-to-leaf path; conditions along the path are ANDed. Example: IF outlook = sunny AND humidity = high THEN play = no. **Advantages:** interpretable, handles nominal and numeric data, fast, no domain knowledge needed. **Disadvantages:** can overfit, unstable to small data changes, greedy (may miss the best tree), biased towards multivalued attributes (information gain).

Weka: Classify > Choose trees > J48 (the Java implementation of C4.5). Parameters: confidenceFactor (default 0.25, smaller means more pruning), minNumObj (default 2 instances per leaf), unpruned, reducedErrorPruning, binarySplits, numFolds. Right-click the result and choose Visualize tree. Other trees: ID3 (nominal only, may need install), RandomTree, REPTree, DecisionStump, RandomForest, LMT.

3.2.4 Covering rules (the covering algorithm, PRISM)

Definition. Covering algorithms build a rule set class by class: for each class, find a rule that covers (matches) instances of that class and as few instances of other classes as possible, remove the covered instances, and repeat. It is called **separate-and-conquer**, in contrast to the divide-and-conquer of decision trees. A tree considers all classes at each split, whereas covering handles one class at a time.

PRISM algorithm (for each class C):

1. Start with all instances in the training set; make an empty rule: IF ? THEN class = C.
2. For each possible attribute-value test, compute rule accuracy p/t, where t is the number of instances the rule covers and p is the number of those that are in class C.
3. Add the test with the highest accuracy p/t (break ties by choosing the larger p, the larger coverage) to the antecedent.
4. Repeat steps 2-3 on the instances covered until the rule is perfect (p/t = 1) or no more attributes remain.
5. Remove the instances covered by this rule, and repeat from step 1 until no instances of class C remain. Then do the next class.

Weather data, class = yes. First rule, candidate accuracies: outlook = sunny 2/5, overcast 4/4, rainy 3/5; temperature = hot 2/4, mild 4/6, cool 3/4; humidity = high 3/7, normal 6/7; windy = false 6/8, true 3/6. Best is outlook = overcast with 4/4, so Rule 1: IF outlook = overcast THEN play = yes (covers 4 instances; remove them). Ten instances remain (5 yes). Candidates now: humidity = normal 4/5 is the best (windy = false 4/6, mild 3/5, rainy 3/5). Within humidity = normal (5 instances) windy = false gives 3/3 (best p/t with largest p), so Rule 2: IF humidity = normal AND windy = false THEN play = yes. Remove the instances covered, and continue until all yes instances are covered.

- **Rules versus trees:** a covering rule set can be more compact than the corresponding tree (trees may replicate subtrees); rule sets are ordered (decision lists) or unordered.
- **Rule quality:** accuracy p/t; also coverage (support). Rules may be pruned (IREP, RIPPER) to avoid overfitting.
- **Weka:** Classify > rules > Prism (nominal data), JRip (RIPPER), PART (rules from partial decision trees), OneR, ZeroR (majority-class baseline), DecisionTable.

3.3 Prediction

3.3.1 Statistical (Bayesian) classification

Bayesian classifiers are statistical classifiers that predict class membership probabilities (the probability that a tuple belongs to a given class). Naive Bayes performs comparably to decision trees and neural networks in many applications.

```
Bayes' theorem:   P(H | X) = P(X | H) * P(H) / P(X)
  H = hypothesis (X belongs to class C); X = observed evidence (attribute values)
  P(H) prior; P(H|X) posterior; P(X|H) likelihood; P(X) evidence.
```

Naive Bayes classifier. Assumes **class-conditional independence**: given the class, attribute values are independent of each other. Then P(X | C_i) = product over k of P(x_k | C_i). Predict the class C_i with the highest P(C_i) * P(X | C_i) (P(X) is constant and can be ignored).

1. Compute prior P(C_i) = (number of training tuples of class C_i) / (total tuples).
2. For each attribute value x_k compute P(x_k | C_i) = count of class-C_i tuples with x_k / count of class-C_i tuples. For numeric attributes assume a Gaussian (normal) density using the class mean and standard deviation.
3. Multiply: P(X | C_i) = P(x_1 | C_i) * P(x_2 | C_i) * ... * P(x_n | C_i).
4. Multiply by the prior and choose the class with the largest value. Normalise to obtain probabilities if needed.

Zero-probability problem and Laplacian correction. If some value never occurs with a class, the product becomes zero. Add 1 to each count (Laplace estimator) so no probability is zero: P(x | C) = (count + 1) / (class count + number of distinct values).

Weather example. New day X: outlook = sunny, temperature = cool, humidity = high, windy = true. Priors: P(yes) = 9/14, P(no) = 5/14. Likelihoods given yes: sunny 2/9, cool 3/9, high 3/9, windy true 3/9. Given no: sunny 3/5, cool 1/5, high 4/5, windy true 3/5. Score(yes) = (9/14) * (2/9)(3/9)(3/9)(3/9) = 0.0053. Score(no) = (5/14) * (3/5)(1/5)(4/5)(3/5) = 0.0206. Normalise: P(yes | X) = 0.0053 / (0.0053 + 0.0206) = 20.5%, P(no | X) = 79.5%. Prediction: play = no.

Strengths: simple, fast, works well with little data, handles missing values (ignore the attribute) and gives probabilities. **Weakness:** independence assumption rarely true; correlated attributes are double counted.

3.3.2 Bayesian (belief) networks

Definition. A Bayesian network (belief network, Bayes net) is a probabilistic graphical model: a **directed acyclic graph (DAG)** whose nodes are random variables and whose arcs show dependence, together with a **conditional probability table (CPT)** for each node giving P(node | its parents). It allows dependence among variables, unlike Naive Bayes (which is a Bayes net where the class is the only parent

of every attribute).

```
Joint probability:  P(x_1, ..., x_n) = product over i of P(x_i | Parents(x_i))
```

- **Training cases:** (1) network structure known and all variables observable: just compute the CPT entries; (2) structure known but some variables hidden: use gradient descent or EM; (3) structure unknown: search over structures (for example K2 hill-climbing, TAN); (4) both structure and variables unknown: hardest.
- **Inference:** compute the posterior of the class variable given observed evidence.
- **Example:** Smoker and FamilyHistory are parents of LungCancer; LungCancer is a parent of PositiveXRay and Emphysema... CPT for LungCancer gives P(LC = yes | FH, Smoker) for each parent combination.
- **Weka:** Classify > bayes > BayesNet (search algorithms K2, TAN, HillClimber; estimator SimpleEstimator) and NaiveBayes (options useKernelEstimator, useSupervisedDiscretization).

3.3.3 Instance-based methods: k-nearest neighbour

Definition. Instance-based (lazy) learners store the training instances and postpone generalisation until a new instance must be classified. **k-nearest neighbour (k-NN)** classifies a new instance by finding the k training instances closest to it and taking the majority class among them (for numeric prediction, the average of their values).

1. Choose k and a distance measure.
2. Compute the distance from the new instance to every training instance.
3. Select the k nearest.
4. Predict the majority class (optionally weight each vote by 1/distance).

```
Euclidean distance:  d(x,y) = sqrt( sum over k of (x_k - y_k)^2 )
Manhattan distance:  d(x,y) = sum over k of |x_k - y_k|
Minkowski (order h): d(x,y) = ( sum over k of |x_k - y_k|^h )^(1/h)
Nominal attributes:  distance 0 if values are equal, 1 if different.
```

Example. Training points: A(1,1) = yes, B(2,2) = yes, C(6,5) = no, D(7,7) = no. New point Q(3,2), k = 3. Distances: to A = sqrt(4 + 1) = 2.24; to B = sqrt(1 + 0) = 1; to C = sqrt(9 + 9) = 4.24; to D = sqrt(16 + 25) = 6.40. The 3 nearest are B, A, C with classes yes, yes, no. Majority = yes. Q is classified as yes.

- **Normalise attributes** (for example min-max) so that large-range attributes do not dominate distance.
- **Choice of k:** small k is noisy (overfits), large k blurs class boundaries; pick by cross-validation. Use odd k for two classes.
- **Advantages:** simple, no training time, handles complex boundaries, easily updated. **Disadvantages:** slow classification (needs all data), memory heavy, sensitive to irrelevant attributes and scaling. Speed-ups: kD-trees, ball trees, editing (condensing) of stored instances, attribute weighting.
- **Weka:** Classify > lazy > IBk (parameters KNN = k, distanceWeighting, crossValidate to choose k, nearestNeighbourSearchAlgorithm), also KStar and LWL.

3.3.4 Linear models

Linear regression models a numeric target as a linear combination of attributes. **Simple linear regression:** y = w0 + w1 * x. **Multiple linear regression:** y = w0 + w1*x1 + w2*x2 + ... + wk*xk. Weights are found by the **least squares method**, minimising the sum of squared differences between actual and predicted values.

```
Least squares for simple regression (x_bar, y_bar are means):
  w1 = sum((x_i - x_bar)(y_i - y_bar)) / sum((x_i - x_bar)^2)
  w0 = y_bar - w1 * x_bar
```

Example (years of experience x, salary y in thousands). Data: (3,30), (8,57), (9,64), (13,72), (3,36), (6,43), (11,59), (21,90), (1,20), (16,83). x_bar = 9.1, y_bar = 55.4. w1 = 3.5 and w0 = 23.6, so salary =

23.6 + 3.5 * years. Predicted salary for 10 years of experience = 23.6 + 35 = 58.6 thousand.

- **Linear models for classification: Logistic regression** models the probability of a class with the logistic (sigmoid) function p = 1 / (1 + e^(-(w0 + w1*x1 + ...))); weights found by maximum likelihood. Perceptron and support vector machines (SVM) also find linear boundaries.
- **Nonlinear relationships:** polynomial regression, or transform variables (log, square) and use a linear model.
- **Evaluating numeric prediction:** mean absolute error (MAE), root mean squared error (RMSE), relative absolute error, root relative squared error, and correlation coefficient between predicted and actual values (higher is better); R-squared.
- **Weka:** Classify > functions > LinearRegression, SimpleLinearRegression, Logistic, SMO (support vector machine), MultilayerPerceptron. The output shows the regression equation and error statistics.

Method	Type	Strengths	Weaknesses
1R / OneR	Rules	Very simple baseline	One attribute only
Decision tree (J48)	Tree	Interpretable, fast	Can overfit, unstable
PRISM / covering rules	Rules	Compact rules, class by class	Can overfit, nominal data
Naive Bayes	Probabilistic	Fast, works with little data	Independence assumption
Bayesian network	Probabilistic	Models dependencies	Structure learning is costly
k-NN (IBk)	Instance-based	No training, flexible boundaries	Slow prediction, scaling-sensitive
Linear / logistic regression	Linear model	Simple, interpretable	Only linear relationships

MODULE IV: Cluster Analysis: Concepts and Methods (Bloom's level L3)

4.1 Basic Issues in Clustering

Definition. Cluster analysis (clustering) groups a set of data objects into clusters so that objects in the same cluster are highly similar (high intra-cluster similarity) and objects in different clusters are dissimilar (low inter-cluster similarity). It is **unsupervised learning**: there are no predefined class labels, so it is also called learning by observation or automatic classification, in contrast to classification which is learning by examples.

Applications: customer (market) segmentation, document and web page grouping, image segmentation, grouping genes or patients, anomaly (outlier) detection, summarisation and preprocessing for other methods, urban planning, grouping suppliers or products in operations and supply chain analytics.

Requirements and issues

- Scalability to large data; ability to handle different attribute types (numeric, nominal, mixed, text).
- Discovery of clusters of arbitrary shape (not only spherical); minimal domain knowledge to set parameters; robustness to noise and outliers.
- Insensitivity to input order; high dimensionality; interpretability and usability; incorporating constraints.
- Deciding the number of clusters and measuring cluster quality.

Distance and similarity measures

```
Euclidean:  d(i,j) = sqrt( sum over k of (x_ik - x_jk)^2 )
Manhattan:  d(i,j) = sum over k of |x_ik - x_jk|
Minkowski:  d(i,j) = ( sum over k of |x_ik - x_jk|^h )^(1/h)
Cosine similarity: sim(x,y) = (x . y) / (||x|| * ||y||)       (used for text)
Jaccard coefficient (binary data): J = q / (q + r + s)   q = both 1; r, s = mismatches
Nominal: simple matching d(i,j) = (p - m) / p   (m = matches among p attributes)
Standardise numeric variables first so that units do not dominate.
```

Categories of clustering methods

Category	Idea	Examples
Partitioning	Divide n objects into k partitions (clusters), improve by relocating objects	k-means, k-medoids (PAM, CLARA, CLARANS)
Hierarchical	Build a hierarchy (tree) of clusters, agglomerative or divisive	AGNES, DIANA, BIRCH, CURE, Chameleon
Density-based	Clusters are dense regions separated by sparse regions; arbitrary shapes	DBSCAN, OPTICS, DENCLUE
Grid-based	Quantise space into a finite number of cells and cluster on the grid	STING, CLIQUE, WaveCluster
Model-based	Fit data to a mathematical or conceptual model	EM (mixture of Gaussians), COBWEB, SOM

Cluster quality: compare against ground truth if available (purity, accuracy in Weka's classes-to-clusters evaluation); otherwise use internal measures such as within-cluster sum of squared errors (SSE) and the silhouette coefficient. Cluster tendency (is the data clusterable?) and the right number of clusters (elbow method, cross-validation) are also assessed.

The Cluster/2 conceptual clustering system

Conceptual clustering forms clusters not by distance alone but by whether objects can be described by a common concept. **CLUSTER/2** (Michalski and Stepp, 1983) is an early conceptual clustering system. It selects k seed objects, then for each seed finds general conjunctive descriptions (complexes, in the form

of attribute-value conditions) that cover that seed but none of the other seeds. The best descriptions according to a quality criterion (such as simplicity, small overlap and coverage) define the clusters; objects are assigned to them, new seeds are chosen (often the most central object of each cluster) and the process repeats until the clustering quality stops improving. Each cluster thus comes with an explicit logical description, which makes the result easy to interpret. It is an early example of clustering with an understandable concept for every cluster, and it is the forerunner of incremental conceptual clustering such as COBWEB.

4.2 Partitioning Methods

4.2.1 k-means

Definition. k-means partitions n objects into k clusters; each cluster is represented by its **centroid** (mean). The user specifies k. It minimises the within-cluster sum of squared errors (SSE).

```
SSE (objective) = sum over clusters i of sum over points p in C_i of  dist(p, m_i)^2
m_i = mean of the objects in cluster C_i
```

1. Choose k initial cluster centres (randomly selected objects).
2. Assign each object to the cluster whose centre is nearest (usually Euclidean distance).
3. Recompute each cluster centre as the mean of the objects now in the cluster.
4. Repeat steps 2 and 3 until assignments no longer change (convergence) or a maximum number of iterations is reached.

1-D example (k = 2). Data: 2, 3, 4, 10, 11, 12, 20, 25, 30. Start with centres m1 = 2 and m2 = 12. Iteration 1: 2, 3, 4 are nearer to 2; the rest nearer to 12. C1 = {2,3,4} mean 3; C2 = {10,11,12,20,25,30} mean 18. Iteration 2 (m1 = 3, m2 = 18): 10 is nearer to 3 (distance 7 versus 8), 11 is nearer to 18 (8 versus 7). C1 = {2,3,4,10} mean 4.75; C2 = {11,12,20,25,30} mean 19.6. Iteration 3 (4.75, 19.6): 11 and 12 move to C1. C1 = {2,3,4,10,11,12} mean 7; C2 = {20,25,30} mean 25. Iteration 4 (7, 25): no changes, so the algorithm stops. Final clusters: {2,3,4,10,11,12} and {20,25,30}.

- **Strengths:** simple, efficient, complexity O(n * k * t) where t = iterations; works well for compact, spherical clusters of similar size.
- **Weaknesses:** k must be specified; sensitive to initial centres (converges to a local optimum; run several times with different seeds or use k-means++); sensitive to noise and outliers (the mean is pulled); only numeric attributes (mean); poor for non-convex shapes and clusters of very different size or density.
- **Variants:** k-modes (categorical data, uses modes), k-prototypes (mixed data), k-medoids (robust to outliers), bisecting k-means.

4.2.2 k-medoids (PAM)

Definition. k-medoids represents each cluster by a **medoid**, the most centrally located actual object in the cluster, instead of the mean. **PAM (Partitioning Around Medoids)** is the basic algorithm.

1. Arbitrarily choose k objects as initial medoids.
2. Assign each remaining object to the nearest medoid.
3. For each medoid m and each non-medoid object o, compute the total cost change of swapping m with o (the cost is the sum of distances of all objects to their medoids, the absolute-error criterion E).
4. If a swap reduces the total cost, make the swap; repeat until no swap improves the cost.

Larger-data variants: **CLARA** (applies PAM to samples), **CLARANS** (randomised search of the graph of possible medoid sets).

Feature	k-means	k-medoids (PAM)
Cluster representative	Centroid (mean), may not be an actual object	Medoid, an actual data object

Feature	k-means	k-medoids (PAM)
Criterion	Sum of squared errors	Sum of absolute distances (absolute error)
Outliers and noise	Sensitive (mean is distorted)	More robust
Data types	Numeric only (needs a mean)	Any data for which a distance exists, including categorical
Complexity per iteration	O(n k t), efficient and scalable	O(k (n - k)^2) per iteration for PAM, costly for large data
Typical use	Large numeric datasets	Small to medium data, noisy data
Result	Can differ run to run with different initial centres	Also depends on initial medoids, but usually more stable

4.2.3 Expectation Maximisation (EM)

Definition. EM is a model-based (probabilistic) clustering method. It assumes data comes from a **mixture of probability distributions** (usually Gaussians), one per cluster, and estimates the parameters (means, standard deviations, mixing weights) that maximise the likelihood of the data. Each object has a probability of belonging to each cluster (**soft or fuzzy clustering**), unlike k-means' hard assignment.

```
Mixture model:  P(x) = sum over k of  w_k * N(x | mu_k, sigma_k)     (sum of w_k = 1)
E-step: for each object x and cluster k, compute the membership probability
        P(k | x) = w_k * N(x | mu_k, sigma_k) / P(x)
M-step: re-estimate the parameters using the weighted objects:
        mu_k = sum(P(k|x) * x) / sum(P(k|x));  w_k = sum(P(k|x)) / n;  sigma_k similarly
```

1. Initialise the parameters (for example random means, equal weights).
2. **E-step (expectation):** compute the probabilities that each object belongs to each cluster with current parameters.
3. **M-step (maximisation):** update the parameters to maximise the expected likelihood.
4. Repeat until the log-likelihood converges (change below a threshold).

Properties: k-means is a special case with equal spherical variances and hard assignment. EM converges to a local maximum of likelihood, can be slow, and is sensitive to the initial values. **Weka EM:** Cluster > EM. Parameters: numClusters (default -1 means the number of clusters is chosen by cross-validation), maxIterations (default 100), minStdDev, seed. The output shows each cluster's prior probability and attribute means and standard deviations, and the log-likelihood.

4.3 Hierarchical Methods

Definition. Hierarchical clustering builds a tree of clusters (a **dendrogram**) showing nested groupings; the user does not need to specify k in advance, and can cut the dendrogram at a chosen level to obtain clusters. Merges and splits are final (no backtracking), which is a limitation.

- **Agglomerative (bottom-up, AGNES):** start with each object as its own cluster; repeatedly merge the two closest clusters until one cluster remains (or a stopping criterion is met). Most common.
- **Divisive (top-down, DIANA):** start with all objects in one cluster; repeatedly split a cluster into smaller ones until each object is alone (or a criterion is met).

Linkage (inter-cluster distance) measures

Method	Distance between clusters	Behaviour
Single linkage (minimum)	Smallest distance between any two objects, one from each cluster	Can create long chains; good for elongated shapes; sensitive to noise
Complete linkage (maximum)	Largest distance between any two objects, one from each cluster	Compact, roughly equal-sized clusters; sensitive to outliers
Average linkage	Mean of all pairwise distances	Compromise

Method	Distance between clusters	Behaviour
Centroid linkage	Distance between cluster means	Simple; can show inversions
Ward's method	Merge that minimises the increase in within-cluster variance	Compact clusters; similar to k-means objective

Example (1-D points). A = 1, B = 2, C = 6, D = 8, E = 11. Pairwise distances: AB 1, CD 2, DE 3, BC 4, AC 5, CE 5, BD 6, AD 7, BE 9, AE 10. **Single linkage:** merge A,B at 1; merge C,D at 2; merge {C,D} with E at 3 (min of CE = 5 and DE = 3); merge {A,B} with {C,D,E} at 4 (BC). **Complete linkage:** merge A,B at 1; merge C,D at 2; {C,D} with E at 5 (max of CE = 5 and DE = 3); then the last merge of {A,B} with {C,D,E} at 10 (max is AE). The merge heights are the dendrogram levels; cutting at a height gives clusters.

Complexity and weaknesses: at least O(n^2) time and memory, so poor for very large data; no undo of earlier merges. **Improved methods:** BIRCH (builds a clustering feature tree, CF-tree, in one scan, then refines), CURE (uses several well-scattered representative points per cluster and shrinks them toward the centre, handles outliers and non-spherical shapes), Chameleon (dynamic modelling: graph partitioning followed by merging using interconnectivity and closeness). **Weka:** Cluster > HierarchicalClusterer (options: linkType single, complete, average, mean, Ward; numClusters; distanceFunction; printNewick); right-click the result to Visualize tree.

4.4 Density-Based and Grid-Based Methods

4.4.1 Density-based clustering: DBSCAN

Idea. Clusters are dense regions of objects separated by regions of low density. Finds clusters of arbitrary shape and identifies noise, with no need to specify k.

- **Epsilon (eps):** the radius of the neighbourhood of an object. **MinPts:** the minimum number of objects required in an eps-neighbourhood to make a dense region.
- **Core object:** has at least MinPts objects (including itself) within eps. **Border object:** not a core object but lies within eps of a core object. **Noise (outlier):** neither core nor border.
- **Directly density-reachable:** q is within the eps-neighbourhood of a core object p. **Density-reachable:** reachable through a chain of directly density-reachable objects. **Density-connected:** p and q are both density-reachable from some object o.

1. Pick an unvisited object p. If p is a core object, start a new cluster.
2. Collect all objects density-reachable from p and add them to the cluster (expand through every core object found).
3. If p is not a core object, mark it as noise (it may later become a border object of another cluster).
4. Repeat until all objects are visited.

Strengths: arbitrary shapes, robust to outliers, no k needed. **Weaknesses:** sensitive to eps and MinPts, struggles with clusters of varying density and with high-dimensional data. **Related: OPTICS** produces a cluster ordering (reachability plot) that works for a range of eps values; **DENCLUE** uses density functions (kernel influence functions). In Weka, DBSCAN and OPTICS are available through a package (install with the Package Manager if not visible under Cluster).

4.4.2 Grid-based clustering

Idea. Quantise the object space into a finite number of cells forming a grid, then perform clustering on the grid. Processing time depends on the number of cells rather than the number of objects, so it is very fast.

- **STING (Statistical Information Grid):** a hierarchical grid of rectangular cells at several resolutions; each cell stores statistics (count, mean, standard deviation, min, max, distribution). Queries are answered top-down, so it is query-independent and supports parallel and incremental update.

- **CLIQUE:** grid-based and density-based; finds clusters in subspaces of high-dimensional data using an Apriori-like approach (dense units in k-1 dimensions imply candidates in k dimensions).
- **WaveCluster:** applies a wavelet transform to the grid feature space; finds clusters of arbitrary shape and removes noise.

Advantages: fast, easy to parallelise. **Disadvantages:** cluster quality depends on grid granularity; cluster boundaries are horizontal or vertical only.

4.5 Conceptual Clustering: COBWEB

Definition. COBWEB is an incremental conceptual (model-based) clustering algorithm that builds a **classification tree** (hierarchy) in which each node is a concept described by probabilistic attribute-value distributions, and each leaf is an object. Objects are processed one at a time. It uses **category utility (CU)** to decide where an object belongs.

```
Category utility of a partition {C_1,...,C_k}:
 CU = (1/k) * sum over clusters C_k of P(C_k) *
      sum over attributes i, values j of [P(A_i=V_ij | C_k)^2 - P(A_i=V_ij)^2]
 Higher CU => better partition (intra-class similarity and inter-class dissimilarity).
```

How COBWEB inserts a new object (starting at the root, going down the tree): at each level it evaluates the following operators and picks the one that gives the highest category utility:

1. **Classify into an existing node:** place the object in the best existing child and recurse.
2. **Create a new node:** make a new child cluster containing only the object.
3. **Merge:** merge the two best children into one.
4. **Split:** split the best child, promoting its children one level.

Strengths: incremental, produces a hierarchy with a probabilistic description of each cluster, works for nominal attributes. **Weaknesses:** depends on input order (merge and split partly compensate), assumes attributes are independent, the tree may become unbalanced, and not suited to large data. **CLASSIT** extends COBWEB to numeric attributes using normal distributions. **Weka Cobweb:** parameters **acuity** (minimum standard deviation for numeric attributes, default 1.0) and **cutoff** (minimum category utility gain to keep a split, small default about 0.0028).

4.6 Weka Experiments for Clustering (k-means, EM, Cobweb)

1. Load data (for example weather.numeric.arff or iris.arff) in Preprocess.
2. Open the Cluster tab. Choose the clusterer: SimpleKMeans, EM, Cobweb, HierarchicalClusterer, or FarthestFirst.
3. Choose a cluster mode: **Use training set**; **Supplied test set**; **Percentage split**; **Classes to clusters evaluation** (choose the class attribute; Weka assigns each cluster the majority class and reports the percentage of incorrectly clustered instances).
4. Click Ignore attributes to exclude the class or ID attributes from clustering.
5. **SimpleKMeans parameters:** numClusters (default 2), seed (random initial centres), maxIterations, distanceFunction (EuclideanDistance default), initializationMethod (random, k-means++, canopy, farthest first), displayStdDevs. The output shows the number of iterations, the **within cluster sum of squared errors**, the cluster centroids (attribute means, nominal modes) and the number and percentage of instances per cluster. Run again with a different seed to test stability, and vary numClusters (2, 3, 4) to compare SSE.
6. Right-click the entry in the result list and choose **Visualize cluster assignments** to plot clusters against any two attributes; use Save to store the clustered data with an extra cluster attribute.
7. For Cobweb, the textual output is the tree in which nodes are numbered and leaves hold instances; use acuity and cutoff to control the number of clusters; right-click > Visualize tree.

Algorithm	Type	Needs k?	Cluster shape	Handles noise	Weka
k-means	Partitioning	Yes	Spherical	Poor	SimpleKMeans
k-medoids	Partitioning	Yes	Spherical	Better	(package)
EM	Model-based	Optional (CV)	Elliptical (Gaussian)	Moderate	EM
Hierarchical	Hierarchical	No (cut tree)	Depends on linkage	Depends	HierarchicalClusterer
DBSCAN	Density	No	Arbitrary	Good	(package)
STING / CLIQUE	Grid	No	Grid-aligned	Good	-
COBWEB	Conceptual	No	Hierarchy	Moderate	Cobweb

MODULE V: Advanced Techniques: Data Mining Software and Applications (Bloom's level L4, L5)

5.1 Text Mining

Definition. Text mining (text data mining, text analytics) is the process of extracting interesting, non-trivial patterns and knowledge from unstructured text. Unlike data mining on structured databases, text must first be converted into a structured numeric form. Tasks: document classification, document clustering, information extraction, sentiment analysis, summarisation, topic modelling, and keyword extraction. Typical business uses: customer review and social media sentiment, email spam filtering, resume screening, news categorisation, complaint routing.

5.1.1 Text preprocessing

1. **Tokenisation:** split text into words (tokens).
2. **Case folding:** convert to lower case.
3. **Stop-word removal:** remove very common words (the, is, of, and).
4. **Stemming:** reduce words to their root by chopping suffixes (Porter stemmer: 'mining', 'mined' becomes 'mine' or 'min'); **lemmatisation:** dictionary-based reduction to the base form ('better' becomes 'good').
5. **Representation:** convert to a document-term matrix.

5.1.2 Extracting attributes (keywords)

In text mining, the attributes (features) of a document are the words or terms it contains. **Bag-of-words (vector space) model:** each document is a vector with one component per vocabulary term, ignoring word order. Components can be binary (present or absent), term frequency (TF) or TF-IDF weight.

```
TF(t,d)   = number of times term t occurs in document d (or normalised by length)
IDF(t)    = log( N / df(t) )    N = number of documents; df(t) = documents with t
TF-IDF    = TF(t,d) * IDF(t)
Cosine similarity of two documents: cos = (d1 . d2) / (||d1|| * ||d2||)
```

Keyword extraction: terms with high TF-IDF are good keywords: frequent in the document but rare in the collection. Words that appear in nearly all documents get an IDF near zero. **Feature (term) selection** reduces the vocabulary using document frequency thresholds, information gain or chi-square. Other methods: n-grams (phrases), RAKE, TextRank, and the use of a controlled vocabulary or thesaurus.

Example. Collection of N = 4 documents; the word 'loan' appears in 2 documents and 'the' in all 4. IDF(loan) = log(4/2) = 0.30 (base 10), IDF(the) = log(4/4) = 0. A document where 'loan' occurs 3 times has TF-IDF(loan) = 3 * 0.30 = 0.90, while 'the' gets 0 no matter how often it occurs, so 'loan' is the keyword.

5.1.3 Structural approaches: parsing and soft parsing

Structural (linguistic) approaches use the grammatical structure of text, not just word counts.

- **Parsing (full parsing):** analyse a sentence according to a formal grammar to produce a complete parse tree (noun phrase, verb phrase, and so on). It gives deep structure but is slow, needs a good grammar and fails on ungrammatical or noisy text.
- **Soft (shallow or partial) parsing:** identifies only certain parts of the structure, such as noun and verb phrases (**chunking**), without producing a full tree. It is faster and more robust on real, messy text (emails, tweets). It is often combined with **part-of-speech (POS) tagging** and **named entity recognition (NER)**: find people, organisations, places, dates, amounts.
- **Uses:** information extraction (filling templates such as who, what, when, where), relation extraction, and creating features (noun phrases) for classification.

5.1.4 Bayesian approach to classifying text

The Naive Bayes method (section 3.3.1) is a standard and very effective text classifier (spam filtering, topic and sentiment classification). A document is treated as a bag of words; class-conditional independence assumes words occur independently given the class.

```
Class score:  P(c | d) is proportional to  P(c) * product over words w in d of P(w | c)
Prior:        P(c) = (documents of class c) / (total documents)
Multinomial estimate with Laplace (add-one) smoothing:
   P(w | c) = ( count(w in class c) + 1 ) / ( total words in class c + |V| )
   |V| = size of the vocabulary
```

Example. Training: spam documents 'buy cheap now' and 'cheap offer now'; non-spam (ham) documents 'meeting at noon' and 'project meeting today'. Vocabulary |V| = 9 (buy, cheap, now, offer, meeting, at, noon, project, today). Total words: spam 6, ham 6. Priors: 0.5 and 0.5. Classify 'cheap meeting now'. Spam: P(cheap) = (2+1)/(6+9) = 0.2, P(meeting) = (0+1)/15 = 0.067, P(now) = 3/15 = 0.2; score = 0.5 * 0.2 * 0.067 * 0.2 = 0.00133. Ham: P(cheap) = 1/15 = 0.067, P(meeting) = (2+1)/15 = 0.2, P(now) = 1/15 = 0.067; score = 0.5 * 0.067 * 0.2 * 0.067 = 0.00044. Normalised, spam = 0.00133/(0.00133 + 0.00044) = 75%, so the document is classified as spam. Without smoothing, the unseen words would make both scores zero.

Multinomial versus Bernoulli model: multinomial uses word counts; Bernoulli uses presence or absence and also counts absent words.

Weka for text mining

- Data: an ARFF with a string attribute for the text and a nominal class attribute (@attribute text string).
- Filter: unsupervised.attribute.StringToWordVector. Options: wordsToKeep (default 1000), TFTransform and IDFTransform (TF-IDF), outputWordCounts, lowerCaseTokens, stopwordsHandler, stemmer (for example LovinsStemmer or Snowball), tokenizer (WordTokenizer, NGramTokenizer), minTermFreq.
- Classify with NaiveBayesMultinomial, NaiveBayes, J48, SMO or Logistic. Use **FilteredClassifier** to apply StringToWordVector inside cross-validation so that the vocabulary is built from training folds only.

5.2 Web Mining

Definition. Web mining applies data mining techniques to discover patterns and knowledge from the World Wide Web: its content, hyperlink structure and usage. Challenges: huge, dynamic, heterogeneous, semi-structured, noisy and of varying quality.

Type	Data used	Goal and techniques
Web content mining	Text, images, tables and multimedia in pages	Classify and cluster pages, extract information, opinion mining; text mining and wrapper induction
Web structure mining	Hyperlink graph between pages	Rank pages, find communities and authorities; PageRank, HITS
Web usage mining	Server logs, clickstreams, sessions, cookies	Understand user navigation and personalise; association rules, sequential patterns, clustering of users and sessions

5.2.1 Classifying web pages

Goal: assign a web page to a category (news, shopping, sports, spam) or topic, for directories, focused crawling, filtering and recommendation.

- **Features:** the page text (bag of words, TF-IDF), title, HTML tags and headings, meta tags, anchor text of incoming links, URL words, words of neighbouring (linked) pages, and page structure.
- **Process:** crawl and fetch pages, clean HTML (remove tags, scripts, navigation), extract text features, select features, train a classifier (Naive Bayes, SVM, decision tree, k-NN) on labelled pages, then

evaluate (accuracy, precision, recall) and classify new pages.

- **Hyperlink information** helps: pages linked to each other tend to share topic (relational and iterative classification); using anchor text of in-links improves accuracy.
- **Challenges:** noisy and boilerplate content, spam and cloaking pages, short pages, multi-topic pages, and the dynamic nature of the web.

5.2.2 Extracting knowledge from the web

- **Web crawling:** automated collection of pages by following links (focused crawlers fetch only pages relevant to a topic).
- **Information extraction and wrappers:** programs that pull structured data (product name, price, rating) from semi-structured HTML; **wrapper induction** learns extraction rules from labelled examples; template detection and web scraping/APIs.
- **Web content mining uses:** price comparison, opinion and review mining, competitor monitoring, knowledge base construction.
- **Web structure mining: PageRank** ranks a page by the importance of the pages linking to it; **HITS** computes **hub** scores (pages that point to good authorities) and **authority** scores (pages pointed to by good hubs).

```
PageRank:  PR(A) = (1 - d) + d * ( PR(T1)/C(T1) + ... + PR(Tn)/C(Tn) )
   T1..Tn = pages linking to A;  C(T) = number of outgoing links of T;
   d = damping factor (usually 0.85)
```

- **Web usage mining steps:** (1) preprocessing (data cleaning of logs, user identification, sessionisation, path completion); (2) pattern discovery (association rules of pages visited together, sequential patterns, clustering of users, classification); (3) pattern analysis and use. **Uses:** website redesign, personalisation, recommendation, e-commerce conversion analysis, fraud detection, caching and prefetching.

5.3 Data Mining Software

Software	Type	Strengths and notes
Weka	Free, open-source (Java, GUI and API)	Broad collection of algorithms, preprocessing and visualisation; ideal for learning; ARFF format; limited for very large data
RapidMiner	Commercial with free tier; visual workflows	Drag-and-drop process design, strong preparation and modelling, deployment; used in industry
KNIME	Open-source, visual workflow nodes	Integrates many tools (Python, R, Weka); data blending and ETL; scalable
Orange	Open-source, visual and Python	Interactive visualisation, good for teaching and exploration
R (Rattle, caret)	Open-source language	Very wide statistical and ML libraries; strong graphics; programming needed
Python (pandas, scikit-learn, NLTK)	Open-source language	Flexible, large ecosystem, deep learning integration; programming needed
SAS Enterprise Miner	Commercial	Enterprise-grade, robust, used in banking and pharma; costly
IBM SPSS Modeler	Commercial	Visual interface, popular in business and social research
Microsoft SQL Server Analysis Services / Azure ML	Commercial	Integrated with Microsoft BI stack and databases
Oracle Data Mining	Commercial	Mining inside the Oracle database, avoids moving data
Tableau, Power BI	BI and visualisation	Dashboards and visual analytics; light-weight modelling; commonly paired with mining tools

How to choose software for a given business problem (evaluation, L5)

- **Problem and data type:** structured, text, web, streaming; size of data (Weka is memory-bound; Spark or databases for big data).
- **Techniques supported:** association, classification, clustering, regression, text mining, time series; availability of preprocessing and evaluation tools.
- **Usability and skills:** GUI/workflow tools suit business analysts; code-based tools (R, Python) suit data scientists.
- **Cost and licensing, vendor support, integration** with existing databases and BI, scalability and performance, deployment and automation, security, visualisation and documentation.

5.4 Data Mining Applications and Issues

Domain	Applications
Retail and marketing	Market basket analysis (association rules), customer segmentation (clustering), targeted promotions, recommendation, customer lifetime value, churn prediction, price optimisation
Banking and finance	Credit scoring and loan default prediction (classification), fraud detection (outlier analysis), customer profiling, stock and risk analysis
Insurance	Claim fraud detection, premium pricing, risk classification
Telecom	Churn prediction, usage pattern clustering, network fault analysis, fraud
Healthcare	Disease diagnosis support, treatment effectiveness, patient clustering, hospital resource planning
Manufacturing and operations	Predictive maintenance, quality control and defect prediction, process optimisation
Supply chain and logistics	Demand forecasting, inventory optimisation, supplier segmentation and risk, route optimisation, delivery delay prediction
E-commerce and web	Recommendation engines, clickstream analysis, web personalisation, search ranking, sentiment analysis of reviews
Human resources and education	Attrition prediction, candidate screening, student performance prediction, course recommendation
Government and public policy	Tax evasion detection, crime pattern analysis, socio-economic segmentation, resource targeting

Issues in data mining. Mining methodology: handling noisy and incomplete data, mining at different abstraction levels, interactive mining, pattern evaluation and interestingness. Performance: efficiency and scalability, parallel and incremental algorithms. Diversity of data types: relational, text, multimedia, streaming, web. **Social and ethical issues:** privacy and security of personal data, data protection laws, bias and fairness in models, transparency and explainability, misuse of discovered knowledge, and the need for ethical data practices and data integrity. Overfitting, spurious patterns (correlation is not causation) and the quality of input data remain important practical risks.

APPENDIX A: Formula Sheet

```
PREPROCESSING
Min-max:   v' = (v - min)/(max - min) * (new_max - new_min) + new_min
Z-score:   v' = (v - mean)/sd              Decimal scaling: v' = v / 10^j
Equal-width bin width = (max - min) / number_of_bins
Pearson r  = sum((a - mean_a)(b - mean_b)) / (n * sd_a * sd_b)
Chi-square = sum((observed - expected)^2 / expected)
Number of cuboids for n dimensions (no hierarchy) = 2^n

STATISTICS
Mean = sum(x)/n    IQR = Q3 - Q1    Outlier fences: Q1 - 1.5*IQR and Q3 + 1.5*IQR
Variance s^2 = sum((x - mean)^2)/(n - 1)

ASSOCIATION RULES
support(A=>B) = count(A and B)/N      confidence(A=>B) = count(A and B)/count(A)
lift(A=>B) = confidence(A=>B)/P(B) = P(A and B)/(P(A)P(B))

CLASSIFICATION
Info(D) = - sum p_i log2 p_i        Gain(A) = Info(D) - Info_A(D)
SplitInfo(A) = - sum (|Dj|/|D|) log2(|Dj|/|D|)     GainRatio(A) = Gain(A)/SplitInfo(A)
Gini(D) = 1 - sum p_i^2
Accuracy = (TP+TN)/(TP+TN+FP+FN)   Precision = TP/(TP+FP)   Recall = TP/(TP+FN)
F = 2PR/(P+R)
PRISM rule accuracy = p/t
Bayes: P(H|X) = P(X|H)P(H)/P(X)    Naive Bayes: P(X|C) = product P(x_k|C)
Laplace: (count + 1)/(class count + number of values)
Euclidean: sqrt(sum (x_k - y_k)^2)     Manhattan: sum |x_k - y_k|
Linear regression: w1 = sum((x - x_bar)(y - y_bar))/sum((x - x_bar)^2)
                   w0 = y_bar - w1*x_bar

CLUSTERING
SSE = sum over clusters sum over points dist(p, centre)^2
EM: P(x) = sum w_k N(x | mu_k, sigma_k)
Category utility (COBWEB) = (1/k) sum P(C_k) sum [P(A_i=V_ij|C_k)^2 - P(A_i=V_ij)^2]

TEXT AND WEB
TF-IDF = TF * log(N/df)     Cosine = (a . b)/(||a|| ||b||)
PageRank PR(A) = (1 - d) + d * sum(PR(T)/C(T)), d = 0.85
```

APPENDIX B: Weka Quick Reference

Task	Where in Weka	Algorithm or filter
Load data	Explorer > Preprocess > Open file	ARFF, CSV
Handle missing values	Preprocess > Filter	ReplaceMissingValues
Discretise	Preprocess > Filter	unsupervised.attribute.Discretize (bins, useEqualFrequency); supervised Discretize
Normalise / standardise	Preprocess > Filter	Normalize, Standardize
Sample or reduce	Preprocess > Filter	Resample, RemovePercentage, Remove
Rank attributes	Select attributes	InfoGainAttributeEval + Ranker; CfsSubsetEval + BestFirst
Association rules	Associate	Apriori, FPGrowth (nominal data)
Baseline	Classify > rules	ZeroR
1R	Classify > rules	OneR
Rule sets	Classify > rules	Prism, JRip, PART, DecisionTable
Decision tree	Classify > trees	J48 (C4.5), REPTree, RandomForest, DecisionStump
Bayesian	Classify > bayes	NaiveBayes, NaiveBayesMultinomial, BayesNet
Nearest neighbour	Classify > lazy	IBk (KNN parameter), KStar
Linear models	Classify > functions	LinearRegression, Logistic, SMO, MultilayerPerceptron
k-means / EM / Cobweb	Cluster	SimpleKMeans, EM, Cobweb, HierarchicalClusterer
Text	Preprocess filter + Classify	StringToWordVector, FilteredClassifier, NaiveBayesMultinomial
Evaluate	Classify > Test options	Use training set, Supplied test set, Cross-validation (10 folds), Percentage split
Visualise	Visualize tab, right-click result list	Scatter plot matrix, Visualize tree, classifier errors, cluster assignments

Reading Weka classification output: 'Correctly Classified Instances' and 'Incorrectly Classified Instances' give accuracy and error rate; the **Kappa statistic** measures agreement beyond chance (1 is perfect, 0 is chance level); MAE and RMSE are error measures; the **Detailed Accuracy By Class** table lists TP rate, FP rate, precision, recall, F-measure, MCC, ROC area and PRC area; the **Confusion Matrix** has rows as actual classes and columns as predicted classes.

APPENDIX C: Glossary

Term	Meaning
Accuracy	Proportion of instances correctly classified.
Agglomerative clustering	Bottom-up hierarchical clustering that repeatedly merges the closest clusters.
Antecedent / consequent	The left (IF) and right (THEN) sides of a rule.
Apriori property	Every subset of a frequent itemset is frequent; used to prune candidates.
ARFF	Attribute-Relation File Format used by Weka.
Association rule	Implication A => B with support and confidence, found in transaction data.
Attribute-oriented induction	Concept description method that generalises task-relevant data using concept hierarchies.
Bayesian network	DAG of variables with conditional probability tables.
Bin / binning	Grouping values into intervals for smoothing or discretisation.
Centroid	Mean point of a cluster (k-means).
Class label	The category attribute to be predicted.
Classification	Predicting a categorical class using a model learnt from labelled data.
Cluster	A group of similar objects.
Concept hierarchy	Ordering of concepts from specific to general (city, state, country).
Confidence	P(B given A) for a rule A => B.
Confusion matrix	Table of actual versus predicted classes.
Core object	In DBSCAN, an object with at least MinPts neighbours within eps.
Cross-validation	Evaluation method that rotates the test fold across k partitions.
Cuboid	A cube at a particular level of summarisation.
Data cube	Multidimensional array of aggregated measures.
Data mart	Departmental subset of a data warehouse.
Data warehouse	Subject-oriented, integrated, time-variant, non-volatile repository for decision support.
Dendrogram	Tree diagram of hierarchical clustering.
Dice	OLAP operation selecting a sub-cube on two or more dimensions.
Dimension	Perspective of analysis in a cube (time, product, location).
Discretisation	Converting numeric values into intervals or categories.
Divisive clustering	Top-down hierarchical clustering that repeatedly splits clusters.
Entropy	Measure of impurity or uncertainty of a class distribution.
EM (Expectation Maximisation)	Iterative method to fit mixture models; soft clustering.
ETL	Extract, transform, load process for warehouses.
Fact table	Central table of measures in a star schema.
Frequent itemset	Itemset with support at least min_sup.
Gain ratio	Information gain normalised by split information (C4.5).
Gini index	Impurity measure used by CART.
Hierarchical clustering	Clustering that builds nested clusters.
Information gain	Reduction in entropy from splitting on an attribute.
Instance	One example (row) in a dataset.
Interestingness	Degree to which a pattern is simple, certain, useful and novel.
Itemset	A set of items.
k-means	Partitioning method using cluster means.

Term	Meaning
k-medoids / PAM	Partitioning method using actual objects as representatives.
k-NN	Nearest-neighbour classifier voting among k closest instances.
KDD	Knowledge discovery in databases, the overall process containing data mining.
Lift	Ratio of observed co-occurrence to that expected under independence.
Medoid	Most central actual object of a cluster.
Naive Bayes	Bayesian classifier assuming attribute independence given the class.
Noise	Random error in data; also objects not in any cluster (DBSCAN).
Normalisation	Scaling attribute values to a common range.
OLAP	Online analytical processing; interactive multidimensional analysis.
OLTP	Online transaction processing; operational database workload.
Outlier	Object that deviates markedly from the rest.
Overfitting	Model too closely fitted to training data, poor on new data.
Pivot	Rotate the axes of a cube view.
Precision / Recall	Fraction of predicted positives that are correct / fraction of actual positives found.
Pruning	Removing parts of a tree to reduce overfitting.
Roll-up	Aggregate to a higher level of a hierarchy or drop a dimension.
Slice	Select one value on one dimension.
Snowflake schema	Star schema with normalised dimension tables.
Star schema	Fact table joined to denormalised dimension tables.
Stemming	Reducing words to root forms.
Stop words	Very common words removed in text mining.
Support	Fraction of transactions containing an itemset or rule.
Supervised learning	Learning from labelled examples.
TF-IDF	Term weight: frequency in the document times inverse document frequency.
Unsupervised learning	Learning without labels (clustering, association).
Weka	Waikato Environment for Knowledge Analysis, open-source mining workbench.
Z-score	Number of standard deviations a value is from the mean.

APPENDIX D: Frequently Asked Questions with Short Answers

Q: What is data mining?

A: Discovery of interesting, previously unknown and potentially useful patterns from large data. It is one step (the core) of the KDD process.

Q: List the steps of KDD.

A: Data cleaning, data integration, data selection, data transformation, data mining, pattern evaluation, knowledge presentation.

Q: Differentiate data mining and OLAP.

A: OLAP is user-driven, verification-oriented summarisation of multidimensional data using roll-up, drill-down, slice, dice, pivot. Data mining is discovery-driven and finds hidden patterns automatically (rules, clusters, classifiers).

Q: Differentiate OLTP and OLAP (data warehouse).

A: OLTP handles current detailed day-to-day transactions for clerks; a data warehouse stores historical, summarised, integrated data for analysts and decision support.

Q: What are the characteristics of a data warehouse?

A: Subject-oriented, integrated, time-variant and non-volatile.

Q: What are OLAP operations?

A: Roll-up, drill-down, slice, dice and pivot.

Q: Why preprocess data?

A: Real data is incomplete, noisy and inconsistent; low-quality data gives low-quality mining results.

Q: How do you handle missing values?

A: Ignore the tuple, fill manually, use a global constant, attribute mean or median, class-wise mean, or the most probable value (regression, Bayes, decision tree).

Q: What is binning?

A: Sorting values and distributing them into bins, then smoothing by bin means, medians or boundaries.

Q: Name normalisation methods.

A: Min-max, z-score and decimal scaling.

Q: What is discretisation?

A: Replacing numeric values with interval or concept labels; it creates concept hierarchies for numeric data.

Q: What is attribute-oriented induction?

A: A concept description method that collects task-relevant data and generalises it by attribute removal and attribute generalisation using concept hierarchies, with threshold control and count accumulation, producing a generalised (prime) relation and rules.

Q: What are interestingness measures?

A: Measures of simplicity, certainty (confidence), utility (support) and novelty used to select useful patterns.

Q: Define support and confidence.

A: Support = fraction of transactions containing A and B. Confidence = P(B given A) = count(A and B) / count(A).

Q: State the Apriori property.

A: All non-empty subsets of a frequent itemset must also be frequent.

Q: Why is lift used?

A: Support and confidence can mislead; lift above 1 means positive correlation, 1 independence, below 1 negative correlation.

Q: Compare classification and prediction.

A: Classification predicts categorical class labels; prediction (regression) predicts continuous values.

Q: How does 1R work?

A: For each attribute make one rule per value predicting the majority class, compute total errors, and choose the attribute with the fewest errors.

Q: How does a decision tree choose the splitting attribute?

A: By an attribute selection measure: information gain (ID3), gain ratio (C4.5) or Gini index (CART).

Q: What is tree pruning?

A: Removing branches to reduce overfitting; prepruning stops early, postpruning trims a full tree.

Q: Compare decision trees and covering rules.

A: Trees use divide-and-conquer over all classes together; covering algorithms (PRISM) use separate-and-conquer, building rules one class at a time and removing covered instances.

Q: Explain Naive Bayes.

A: Applies Bayes' theorem with the assumption of class-conditional independence of attributes; picks the class with the highest prior times product of attribute likelihoods.

Q: How does a Bayesian network differ from Naive Bayes?

A: A Bayesian network allows dependencies among attributes through a DAG and conditional probability tables; Naive Bayes assumes all attributes depend only on the class.

Q: What is k-NN?

A: A lazy learner that classifies a new instance by the majority class of its k nearest training instances.

Q: Explain linear regression.

A: Fits y = w0 + w1x1 + ... by least squares to predict numeric values.

Q: Compare k-means and k-medoids.

A: k-means uses cluster means (centroids) and squared error, fast but sensitive to outliers and numeric-only; k-medoids uses actual objects as medoids and absolute error, more robust but slower.

Q: Compare hierarchical and partitioning clustering.

A: Partitioning needs k in advance and gives one flat partition; hierarchical builds a dendrogram, no k needed, but cannot undo merges and is costly for large data.

Q: What is EM clustering?

A: Probabilistic clustering that fits a mixture of distributions by alternating an expectation step (membership probabilities) and a maximisation step (parameter updates).

Q: What is DBSCAN?

A: Density-based clustering using eps and MinPts that finds arbitrary-shaped clusters and noise.

Q: What is COBWEB?

A: Incremental conceptual clustering that builds a classification tree using category utility, with insert, new-node, merge and split operators.

Q: What is text mining and TF-IDF?

A: Extracting knowledge from unstructured text; TF-IDF weights a term by its frequency in a document times the log of inverse document frequency in the collection.

Q: What is web mining?

A: Mining of web content, web structure (links) and web usage (logs).

Q: Name some data mining software.

A: Weka, RapidMiner, KNIME, Orange, R, Python (scikit-learn), SAS Enterprise Miner, IBM SPSS Modeler.

Q: What are the ethical issues in data mining?

A: Privacy, security, bias and fairness, transparency, misuse of results and data integrity.